MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

VREMERE ADRIAN, FAF-232

Report

Laboratory Work n.3.2

Analog Signal Acquisition and Signal Conditioning

on Embedded Systems

Checked by:
Martîniuc Alexei, *univ. asist.*
FCIM, UTM

Chişinău – 2026

1. Domain Analysis

1.1. Objective of the Laboratory Work

The objective of this laboratory work is to design, implement, and validate a signal conditioning pipeline for dual-sensor temperature monitoring on an Arduino Mega 2560 microcontroller. Building upon the sensor acquisition and threshold alerting foundations established in the previous laboratory, this work introduces a three-stage signal conditioning chain — saturation clamping, median filtering, and exponentially weighted moving average (EWMA) smoothing — that transforms raw sensor readings into clean, reliable temperature values suitable for threshold-based decision-making. The system reads temperature data from two distinct sensor types — an analog NTC thermistor and a digital DS18B20 — applies independent conditioning pipelines to each sensor channel, and presents all intermediate and final values through both an LCD display and structured STDIO reporting.

The key differentiator of this laboratory from the previous work is the explicit signal conditioning pipeline inserted between raw data acquisition and threshold alerting. Where the previous lab operated directly on converted temperature values, this lab demonstrates that raw sensor data in real-world embedded systems is inherently noisy and must be systematically conditioned before it can be trusted for decision-making. The three conditioning stages address different noise characteristics: saturation clamping rejects physically impossible outliers, median filtering removes impulsive salt-and-pepper noise spikes, and EWMA smoothing attenuates high-frequency random noise while preserving the underlying temperature trend.

The learning objectives include understanding the theoretical basis and practical implementation of each conditioning stage, analyzing the trade-offs between noise rejection and signal latency introduced by filtering, implementing modular signal conditioning libraries that can be independently configured per sensor channel, using FreeRTOS preemptive multitasking with mutex and semaphore synchronization to orchestrate the acquisition-conditioning-display pipeline, and formatting floating-point sensor data for display using the `dtostrf` function (since the AVR implementation of `printf` does not support the `%f` format specifier).

1.2. Problem Definition

The task requires the development of a microcontroller-based temperature monitoring application with a configurable signal conditioning pipeline. The system must periodically acquire temperature data from two sensors: one analog sensor (NTC thermistor) read via the built-in 10-bit ADC, and one digital sensor (DS18B20) read via the OneWire protocol. The acquisition period must be configurable, with a default of 50 milliseconds to ensure adequate temporal resolution for the conditioning filters.

The acquired raw temperature data must pass through a three-stage conditioning pipeline before being used for threshold evaluation. The first stage is saturation clamping, which restricts the temperature value to a physically valid range (for example, −40°C to +125°C), rejecting any

readings that fall outside this range by clamping them to the nearest boundary. The second stage is median filtering, which maintains a sliding window of the N most recent saturated samples (default $N = 5$), sorts them, and outputs the middle value, thereby eliminating impulsive noise spikes that affect only one or two samples. The third stage is EWMA smoothing, which applies an exponentially weighted moving average with a configurable smoothing factor α (default $\alpha = 0.3$) to attenuate residual high-frequency noise while tracking the true temperature trend.

Each sensor channel must have its own independent conditioning pipeline instance, allowing different filter parameters to be configured per sensor if desired. Threshold alerting with hysteresis and software debounce must operate on the conditioned output values (after all three stages), not on the raw readings. This ensures that alert decisions are based on clean, filtered data rather than noisy raw samples.

The system must provide two output channels. An I2C LCD display shows the current conditioned temperature readings from both sensors and the alert status, alternating between information pages. Simultaneously, a structured report is printed to the serial terminal via STDIO at a configurable interval (every 2 seconds), providing detailed diagnostic information including raw values, saturated values, median-filtered values, EWMA-smoothed values, alert states, and cumulative statistics. All floating-point values in the STDIO output are formatted using `dtostrf` due to the AVR platform's lack of `%f` support in `printf`.

The application architecture follows a modular design with clearly separated concerns, orchestrated by the FreeRTOS scheduler. The sensor acquisition task (highest priority) reads both sensors and stores the raw data. The conditioning task (medium priority) applies the three-stage pipeline and evaluates thresholds on the conditioned output. The display and reporting task (lowest priority) presents intermediate and final values to the user. Inter-task synchronization uses binary semaphores for event signaling and mutexes for shared data protection.

1.3. Task Statement

> *Sarcina de laborator — Achiziție semnal analogic*
>
> *Pentru un senzor analogic sau digital de temperatură, să se realizeze o aplicație pentru MCU care va achiziționa periodic semnalul de la senzorul selectat dintr-o listă de senzori, va filtra semnalul și va afișa valorile intermediare și cea curentă prin STDIO fie către un afișor LCD, fie către o interfață serială.*
>
> *Varianta de realizare: implementarea unui sistem de monitorizare a doi senzori, unul analogic și unul digital, cu afișarea valorilor și alertelor de la ambii senzori pe display LCD.*

The task statement requires the implementation of a periodic signal acquisition system for temperature sensors with signal filtering and display of intermediate values. The selected implementation variant extends this to a dual-sensor configuration combining an analog NTC thermistor and a digital DS18B20 sensor, with both sensor channels independently filtered through a three-stage conditioning pipeline. The system displays raw, intermediate, and final conditioned values

through both an LCD display and a serial STDIO interface, providing full visibility into the conditioning process at every stage.

1.4. Used Technologies

Signal Conditioning in Embedded Systems

Raw sensor data in embedded systems is rarely suitable for direct use in decision-making. Analog sensors are subject to thermal noise, quantization error, electromagnetic interference, and component drift, while digital sensors can produce occasional erroneous readings due to communication glitches, timing violations, or internal conversion errors. Signal conditioning refers to the systematic chain of processing operations applied to raw sensor data to improve its quality, reliability, and suitability for downstream processing.

A well-designed conditioning pipeline typically includes multiple stages, each targeting a specific class of noise or artifact. The ordering of these stages matters: coarse rejection of impossible values should precede fine-grained filtering, and nonlinear filters (such as median) should precede linear smoothing filters (such as EWMA) to prevent outliers from corrupting the smoother's internal state. In this laboratory, the three-stage pipeline — saturation, median filter, EWMA — follows this principle and represents a practical, computationally efficient conditioning chain suitable for resource-constrained microcontrollers.

Saturation (Signal Clamping)

Saturation, also known as signal clamping, is the simplest and most fundamental conditioning operation. It restricts the input value to a predefined valid range by replacing any value below the minimum with the minimum and any value above the maximum with the maximum. In this laboratory, the saturation range is set to $[-40, +125]$°C, corresponding to the operational limits of the sensors used.

The purpose of saturation is twofold. First, it rejects physically impossible readings that may result from sensor faults, disconnected wiring, or ADC overflow conditions. For example, an open-circuit NTC thermistor produces an ADC reading of 1023, which the Steinhart-Hart equation maps to a temperature near -273°C — a physically meaningless value that, if propagated through the pipeline, could corrupt downstream filter states. Second, saturation prevents the propagation of NaN (Not a Number) or infinity values that can arise from division-by-zero conditions in the temperature conversion equations. By clamping the output to a finite, bounded range, saturation ensures that all subsequent pipeline stages receive numerically well-behaved inputs. Despite its simplicity, saturation is an essential first line of defense in any signal conditioning chain.

Median Filter (Salt-and-Pepper Noise Removal)

The median filter is a nonlinear digital filter that is particularly effective at removing impulsive noise — also known as salt-and-pepper noise — from a signal. Unlike linear filters (such as the moving average), which spread the energy of a noise spike across multiple output samples,

the median filter completely eliminates isolated outliers while preserving the underlying signal characteristics, including sharp edges and step changes.

The median filter operates by maintaining a sliding window of the N most recent input samples. At each time step, the filter sorts the values within the window and outputs the middle element (the median). For a window size of $N = 5$, this means that up to 2 corrupted samples out of 5 can be present without affecting the output — the median will still correspond to a valid measurement. This property makes the median filter ideal for rejecting sensor glitches that affect only a single sample or a small number of consecutive samples.

The choice of window size N involves a trade-off between noise rejection capability and signal latency. A larger window tolerates more simultaneous outliers and provides better noise suppression, but introduces greater delay because the output reflects the median of a larger historical buffer. For temperature monitoring applications where the physical process changes slowly relative to the sampling rate, a window of $N = 5$ provides excellent impulse rejection with minimal latency (approximately $N/2 = 2.5$ sample periods of group delay). An odd window size is preferred because it guarantees a unique middle element, avoiding the need to average two central values as would be required with an even-sized window.

The computational cost of the median filter is dominated by the sorting step. For a window of size N, a comparison-based sort requires $O(N \log N)$ operations in the general case, though for the small window sizes typical in embedded applications ($N = 3$ to $N = 7$), simple insertion sort with $O(N^2)$ complexity is often preferred due to its low overhead and cache-friendly access pattern. The memory requirement is modest: only N samples need to be stored in a circular buffer.

Exponentially Weighted Moving Average (EWMA)

The Exponentially Weighted Moving Average (EWMA) is a first-order infinite impulse response (IIR) low-pass filter that smooths a noisy signal by computing a weighted combination of the current input sample and the previous output. The EWMA is defined by the recursive formula:

$$y_n = \alpha \cdot x_n + (1 - \alpha) \cdot y_{n-1}$$

where y_n is the filter output at time step n, x_n is the current input sample, y_{n-1} is the previous output, and α is the smoothing factor with $0 < \alpha \leq 1$. The parameter α controls the balance between responsiveness and smoothness: a larger α (closer to 1) makes the filter respond more quickly to input changes but provides less noise attenuation, while a smaller α (closer to 0) produces a smoother output but introduces greater lag in tracking genuine signal changes. In this laboratory, $\alpha = 0.3$ is used, providing a good compromise between noise rejection and response time for temperature monitoring.

The EWMA can be interpreted as a weighted sum of all past input samples, where the weight of each sample decreases exponentially with its age. The effective memory of the filter — the number of past samples that contribute significantly to the output — is approximately $2/\alpha$ samples. For $\alpha = 0.3$, this corresponds to roughly 6–7 samples, meaning the filter output reflects a

weighted average of the most recent 6–7 readings.

Compared to a simple moving average (SMA) of equivalent smoothing, the EWMA offers two significant advantages for embedded systems. First, it requires only a single state variable (the previous output y_{n-1}) rather than a buffer of N past samples, making it extremely memory-efficient. Second, its computational cost is constant — just one multiplication and one addition per sample — regardless of the effective averaging window. These properties make the EWMA an ideal smoothing filter for resource-constrained microcontrollers where both memory and CPU cycles are at a premium.

The EWMA is initialized with the first valid input sample ($y_0 = x_0$), which ensures that the filter output immediately reflects the actual signal level rather than ramping up from zero. After initialization, the filter converges to the steady-state signal level within approximately $3/\alpha$ samples.

Analog-to-Digital Conversion (ADC)

Analog-to-Digital Conversion is the process of translating a continuous analog voltage signal into a discrete digital representation that a microcontroller can process. The Arduino Mega 2560 features a 10-bit successive approximation ADC with 16 multiplexed input channels. The 10-bit resolution means the ADC maps the input voltage range (0 V to 5 V by default) into 1024 discrete levels, yielding a resolution of approximately 4.88 mV per step.

The ADC conversion process begins when the firmware triggers a conversion on the selected channel. The successive approximation register (SAR) algorithm performs a binary search through the possible output codes, comparing the input voltage against an internally generated reference at each step. On the ATmega2560, the ADC clock is derived from the system clock through a prescaler, and a full conversion requires 13 ADC clock cycles (25 cycles for the first conversion after enabling the ADC). At the default prescaler setting of 128 (yielding a 125 kHz ADC clock from the 16 MHz system clock), a single conversion takes approximately 104 microseconds.

For temperature measurement with an NTC thermistor, the ADC reading represents the voltage at the midpoint of a resistive voltage divider formed by the thermistor and a fixed series resistor. The raw ADC value is first converted to resistance using the voltage divider equation, and subsequently to temperature using the Steinhart-Hart Beta parameter equation:

$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{B} \cdot \ln\left(\frac{R_{NTC}}{R_0}\right)$$

where T is the absolute temperature in Kelvin, T_0 is the reference temperature (typically 298.15 K = 25°C), B is the Beta coefficient of the thermistor (3950 K for the sensor used in this lab), R_{NTC} is the measured thermistor resistance, and R_0 is the nominal resistance at T_0 (10 kΩ). The resistance is calculated from the ADC reading using:

$$R_{NTC} = R_{series} \cdot \frac{ADC}{ADC_{max} - ADC}$$

where R_{series} is the fixed series resistor (10 kΩ), ADC is the raw digital reading, and ADC_{max} is the maximum ADC value (1023 for 10-bit resolution). The resulting temperature in

Kelvin is converted to Celsius by subtracting 273.15. On the AVR platform, the floating-point result is formatted for display using `dtostrf` rather than `printf` with `%f`, as the AVR C library's `printf` implementation omits floating-point support to reduce code size.

OneWire Communication Protocol

OneWire is a proprietary communication protocol developed by Dallas Semiconductor (now Maxim Integrated / Analog Devices) that enables bidirectional data exchange between a master device and one or more slave devices over a single data wire plus ground. The protocol is particularly significant in embedded temperature sensing because it allows the DS18B20 digital temperature sensor to communicate its readings using only one GPIO pin, minimizing the wiring complexity of multi-sensor installations.

The OneWire protocol operates at TTL voltage levels and uses an open-drain bus topology with a pull-up resistor (typically 4.7 kΩ) to VCC. Communication is initiated exclusively by the master, which drives the bus low to generate timing slots. The protocol defines several fundamental operations: Reset/Presence pulse (the master drives the bus low for at least 480 μs, then releases it; slave devices respond by pulling the bus low for 60–240 μs), Write Time Slot (the master drives the bus low and either releases it quickly for a logical '1' or holds it low for a logical '0'), and Read Time Slot (the master briefly drives the bus low, then samples the bus state; the slave holds the bus low for a '0' or releases it for a '1').

Each slave device on the OneWire bus has a unique 64-bit ROM code that serves as its address. For temperature measurement, the master issues a Convert T command (0x44), which instructs the DS18B20 to begin its internal analog-to-digital conversion. The conversion time depends on the configured resolution: 93.75 ms at 9-bit through 750 ms at 12-bit resolution. After conversion completes, the master reads the scratchpad memory containing the temperature result.

I2C Communication Protocol

Inter-Integrated Circuit (I2C) is a synchronous, multi-master, multi-slave serial communication protocol that uses two bidirectional open-drain lines: Serial Data (SDA) and Serial Clock (SCL). Developed by Philips Semiconductor (now NXP) in 1982, I2C has become one of the most widely used communication interfaces in embedded systems for connecting low-speed peripheral devices such as sensors, EEPROMs, real-time clocks, and LCD controllers.

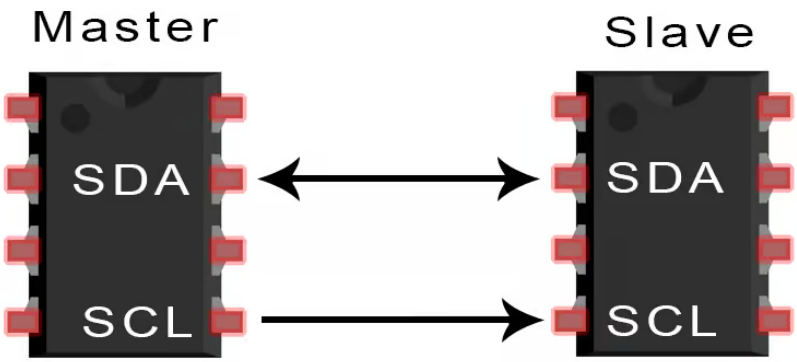

***Figure 1.1** – I2C communication protocol timing diagram*

In this laboratory, I2C is used to communicate with the LCD 1602 display module through a PCF8574 I/O expander chip. The LCD module is configured at I2C address 0x27, and communication occurs at the standard 100 kHz clock speed. The Arduino Mega 2560 provides dedicated I2C pins: SDA on pin 20 and SCL on pin 21. Each I2C transaction follows a defined sequence: the master generates a START condition, transmits the 7-bit slave address followed by a read/write bit, waits for an acknowledgment from the addressed slave, transfers one or more data bytes, and generates a STOP condition. The LiquidCrystal_I2C library abstracts these low-level protocol details, providing a simple API for text display operations, as shown in *Figure 1.1*.

FreeRTOS Real-Time Operating System

FreeRTOS is a real-time operating system (RTOS) kernel for embedded microcontrollers that provides preemptive multitasking, inter-task synchronization primitives, and deterministic scheduling. In this laboratory, FreeRTOS manages three concurrent tasks that together implement the sensor monitoring pipeline: acquisition, conditioning (including the three-stage filter chain and threshold evaluation), and display/reporting.

The FreeRTOS scheduler uses a priority-based preemptive algorithm: at any given moment, the highest-priority task that is ready to run receives CPU time. When a higher-priority task becomes ready (for example, when a semaphore it was waiting on is given), the scheduler immediately preempts the running lower-priority task and switches context to the higher-priority one. This mechanism ensures that time-critical operations such as sensor acquisition are never delayed by lower-priority work such as LCD display updates.

The synchronization primitives used in this lab include binary semaphores and mutexes. A binary semaphore acts as a lightweight signaling mechanism: the acquisition task *gives* the semaphore after completing a reading cycle, and the conditioning task *takes* the semaphore to wake up and process the new data. A mutex (mutual exclusion semaphore) protects the shared data structures accessed by multiple tasks, ensuring that only one task can read or modify the shared state at any time. FreeRTOS mutexes provide priority inheritance: if a low-priority task holds the mutex and a high-priority task attempts to acquire it, the low-priority task temporarily inherits the higher priority, preventing the pathological condition known as priority inversion.

Hysteresis-Based Threshold Detection

Hysteresis is a signal conditioning technique that uses two distinct thresholds — an upper (high) threshold and a lower (low) threshold — to determine state transitions, preventing rapid oscillation (chattering) when the signal hovers near a single threshold boundary. In the context of temperature monitoring, the alert state activates when the temperature rises above the high threshold (e.g., 30°C) and deactivates only when the temperature drops below the low threshold (e.g., 28°C).

A critical design decision in this laboratory is that hysteresis-based threshold detection operates on the *conditioned* output values — that is, the values that have already passed through the saturation, median filter, and EWMA stages — rather than on the raw sensor readings. This ensures that threshold decisions are based on clean, filtered data, significantly reducing the risk of false alerts caused by transient noise spikes or sensor glitches. The debounce mechanism further strengthens robustness by requiring N consecutive threshold crossings (configurable, default 5) before confirming a state transition, ensuring that even residual noise in the conditioned signal does not trigger spurious alerts.

1.5. Hardware Components

Arduino Mega 2560

The Arduino Mega 2560 is the central microcontroller development board used in this laboratory. It is based on the ATmega2560 AVR microcontroller, which features a 16 MHz clock frequency, 256 KB of Flash program memory, 8 KB of SRAM, and 4 KB of EEPROM. The board provides 54 digital I/O pins (15 of which support PWM output), 16 analog input pins with 10-bit ADC resolution, 4 hardware UART serial ports, an SPI interface, and an I2C interface.

For this laboratory, the Arduino Mega 2560 serves multiple roles: it reads the analog NTC thermistor through its ADC (pin A0), communicates with the DS18B20 via a digital GPIO pin (pin 2) using the OneWire protocol, drives an I2C LCD display through its dedicated SDA/SCL pins (pins 20/21), controls three indicator LEDs through digital GPIO pins (pins 8, 9, 10), and runs the FreeRTOS scheduler to manage the acquisition-conditioning-display task pipeline. The board's 8 KB SRAM is essential for running FreeRTOS with multiple tasks (each requiring its own stack allocation) and for maintaining the median filter buffers and EWMA state variables across both sensor channels.

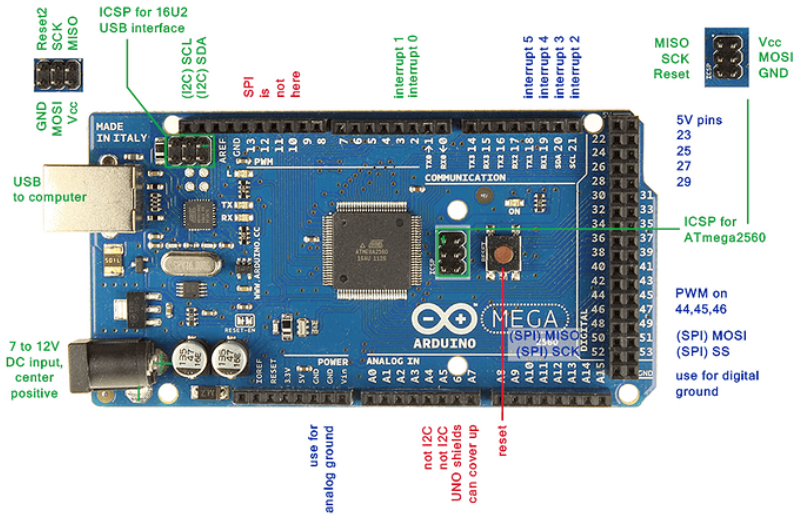

Figure 1.2 *– Arduino Mega 2560 development board*

NTC Thermistor (Analog Temperature Sensor)

An NTC (Negative Temperature Coefficient) thermistor is a resistive temperature sensor whose electrical resistance decreases as temperature increases. The thermistor used in this laboratory has a nominal resistance of 10 kΩ at 25°C and a Beta coefficient (B value) of 3950 K. These parameters define the resistance-temperature characteristic curve through the Steinhart-Hart Beta equation.

The NTC thermistor is connected in a voltage divider configuration with a 10 kΩ fixed series resistor. The midpoint voltage of this divider is fed to the Arduino's analog input pin A0. As the temperature changes, the thermistor resistance changes, altering the voltage division ratio and producing a corresponding change in the ADC reading. The analog nature of this sensor makes it particularly susceptible to electrical noise, quantization effects, and occasional erroneous readings caused by electromagnetic interference — precisely the types of artifacts that the three-stage signal conditioning pipeline is designed to address.

Figure 1.3 *– NTC thermistor (10K, Beta=3950)*

DS18B20 Digital Temperature Sensor

The DS18B20 is a digital temperature sensor manufactured by Maxim Integrated that provides 9-to-12-bit temperature measurements over a single-wire digital interface. Unlike the NTC thermistor, which outputs an analog voltage that must be externally converted, the DS18B20 performs its own internal analog-to-digital conversion and communicates the result digitally, eliminating analog noise in the communication path. However, the DS18B20 can still produce occasional erroneous readings due to OneWire timing violations or CRC failures, making signal conditioning equally important for this digital sensor channel.

The DS18B20 measures temperatures from $-55°C$ to $+125°C$ with a configurable resolution. At 10-bit resolution (used in this lab), the sensor provides $0.25°C$ precision with a conversion time of approximately 187.5 ms. A 4.7 kΩ pull-up resistor on the DQ line is required because the OneWire bus uses an open-drain topology.

Figure 1.4 *– DS18B20 digital temperature sensor (TO-92 package)*

LCD 1602 with I2C Interface

The LCD 1602 is a character liquid crystal display module capable of showing 16 characters per line across 2 lines. When equipped with a PCF8574-based I2C backpack module, the display communicates with the microcontroller over just two wires (SDA and SCL) instead of requiring 6–10 parallel data pins. The I2C address is typically 0x27.

In this laboratory, the LCD serves as the primary real-time display for conditioned temperature readings and alert status. The display alternates between information pages, showing current analog and digital temperatures (after the full conditioning pipeline) along with the overall system status. The 16-character line width constrains the information density, requiring careful formatting with `dtostrf` to present floating-point temperature values in a compact form.

Figure 1.5 *– LCD 1602 display with I2C backpack module*

LEDs with Current-Limiting Resistors

Three LEDs provide immediate visual feedback on the system state. Each LED is connected to a digital GPIO pin through a 220 Ω current-limiting resistor, which restricts the forward current to approximately 15 mA at the 5 V logic level. The green LED (pin 8) indicates normal system operation with both sensors below their respective thresholds. The red LED (pin 9) indicates that the analog sensor has triggered a threshold alert based on its conditioned output. The yellow LED (pin 10) indicates that the digital sensor has triggered a threshold alert. During debounce transitions, the corresponding LED blinks to indicate that a state change is being validated but not yet confirmed.

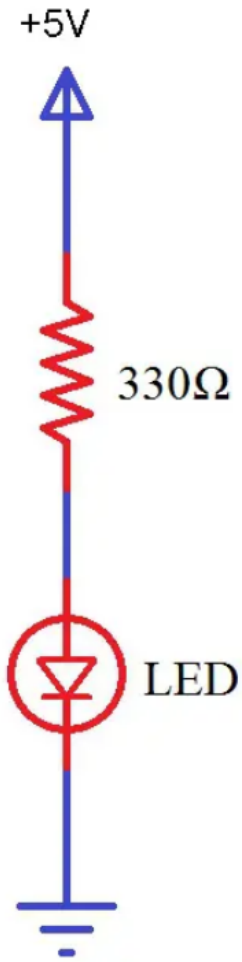

Figure 1.6 *– Standard 5mm LEDs (green, red, yellow)*

Resistors

Several resistors are used in the circuit, each serving a distinct purpose. A 10 kΩ resistor forms the upper half of the voltage divider for the NTC thermistor, creating a known reference against which the thermistor's variable resistance produces a measurable voltage at the ADC input. A 4.7 kΩ pull-up resistor on the DS18B20 data line provides the required pull-up voltage for the open-drain OneWire bus, ensuring reliable signal levels during the precisely timed communication slots. Three 220 Ω resistors limit the current through the indicator LEDs to safe operating levels, protecting both the LEDs and the microcontroller GPIO pins from excessive current draw.

1.6. Software Components

PlatformIO

PlatformIO is a professional open-source ecosystem for embedded development that integrates with Visual Studio Code as an extension. It provides comprehensive project management,

automatic library dependency resolution, multi-board and multi-environment support, and unified build/upload/monitor tools. In this laboratory, PlatformIO manages the compilation environment for the Arduino Mega 2560, automatically resolves library dependencies (FreeRTOS, OneWire, DallasTemperature, LiquidCrystal_I2C), and provides the serial monitor for observing STDIO output. The PlatformIO project configuration (`platformio.ini`) defines a dedicated environment for Lab 3.2 with the required build flags and library dependencies, enabling the project to maintain multiple lab configurations simultaneously.

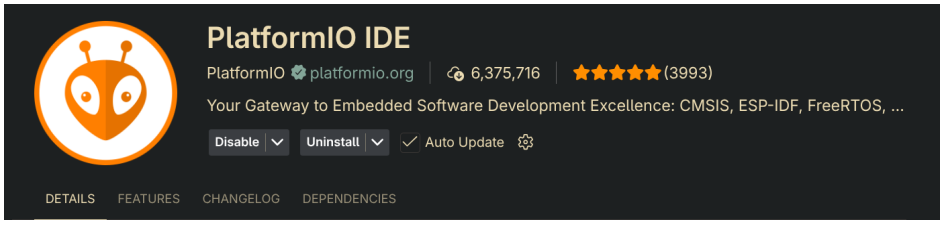

***Figure 1.7** – PlatformIO IDE in VS Code with Lab 3.2 project*

Wokwi Simulator

Wokwi is an embedded systems simulator available both as a web application and as a VS Code extension. It supports Arduino, ESP32, Raspberry Pi Pico, and other popular platforms, providing virtual components including LEDs, buttons, LCDs, temperature sensors (NTC thermistors and DS18B20), and serial terminals. Wokwi enables testing of the complete application — including the signal conditioning pipeline — in a simulated environment before deploying to physical hardware. The simulator's ability to inject controlled temperature changes and noise patterns is particularly valuable for validating the behavior of the saturation, median filter, and EWMA stages under various input conditions.

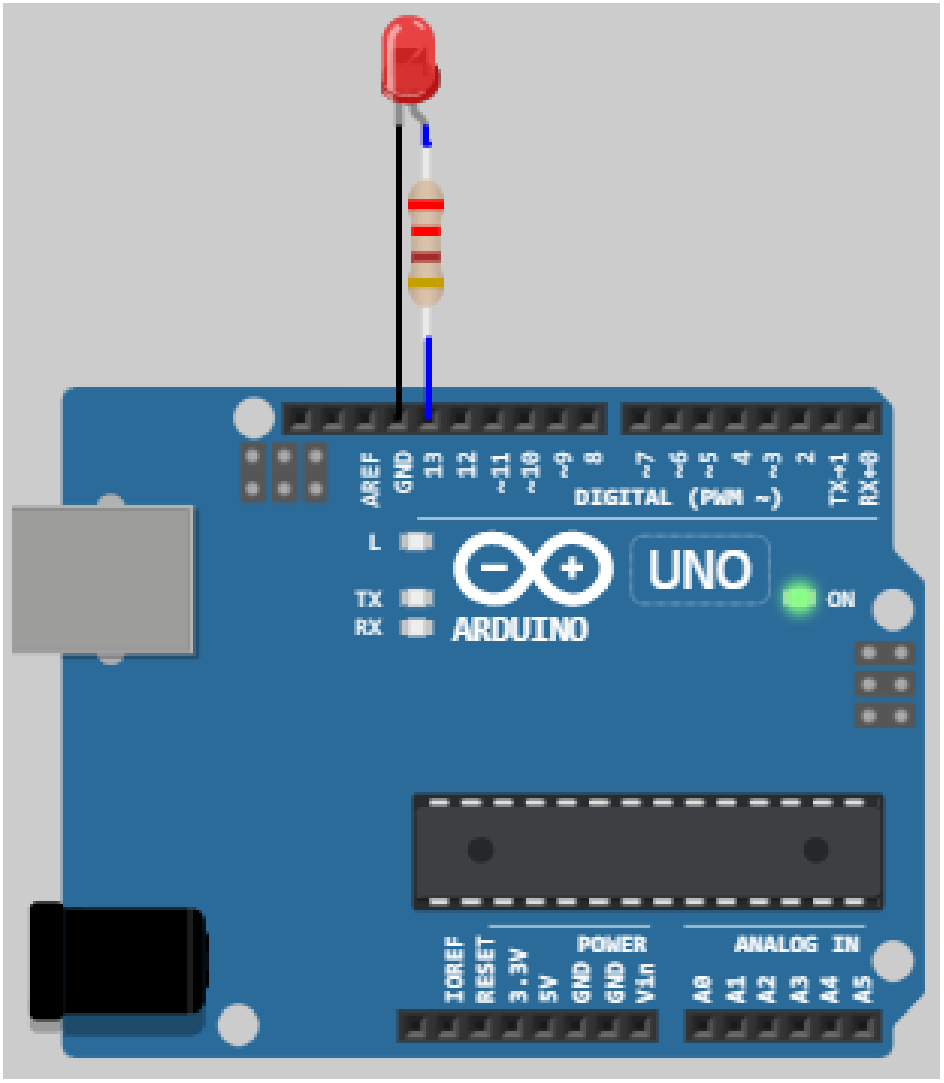

Figure 1.8 *– Wokwi simulator running the Lab 3.2 circuit*

1.7. Case Study: Signal Conditioning in Industrial Process Control

The three-stage signal conditioning pipeline implemented in this laboratory — saturation clamping, median filtering, and EWMA smoothing — directly mirrors techniques used in industrial process control systems where measurement reliability is paramount for safe and efficient operation.

In pharmaceutical manufacturing, regulatory frameworks such as FDA 21 CFR Part 211 mandate continuous environmental monitoring with documented evidence that temperature-sensitive products are stored within specified ranges. Pharmaceutical cold chain monitoring systems deploy arrays of temperature sensors across storage facilities, and raw sensor data must be conditioned before it is compared against regulatory thresholds. A single erroneous sensor reading that triggers a false out-of-specification alert can require discarding an entire batch of medication — a potentially multi-million-dollar loss. Saturation clamping prevents sensor faults (such as a disconnected probe reading $-273°C$) from corrupting the monitoring record. Median filtering removes transient spikes caused by electromagnetic interference from nearby HVAC equipment or motor drives. EWMA smoothing tracks the true storage temperature while filtering out high-frequency noise, ensuring

that threshold alerts reflect genuine temperature excursions rather than measurement artifacts.

Food safety monitoring presents similar challenges. HACCP (Hazard Analysis and Critical Control Points) regulations require that critical control points — such as cooking temperatures, refrigeration units, and blast chillers — be continuously monitored with documented alarm response procedures. In a commercial food processing facility, temperature sensors are exposed to steam, vibration, and electrical noise from high-power cooking equipment. The conditioning pipeline ensures that the monitoring system alerts operators to genuine temperature violations while suppressing the nuisance alarms that can lead to alarm fatigue and, paradoxically, reduced safety compliance.

HVAC (Heating, Ventilation, and Air Conditioning) building management systems represent yet another application domain. Modern building automation controllers read hundreds of temperature, humidity, and pressure sensors distributed throughout a facility. The raw sensor data is conditioned through configurable filter chains before being fed to PID control loops and alarm engines. An unconditioned noisy temperature signal driving a PID controller would cause the HVAC actuators (dampers, valves, compressors) to oscillate rapidly, increasing mechanical wear, energy consumption, and occupant discomfort. The EWMA filter's ability to smooth high-frequency noise while tracking slow temperature trends makes it a standard component in commercial building controllers, and its minimal computational and memory requirements (a single state variable per channel) allow it to be deployed across hundreds of sensor channels simultaneously on resource-constrained embedded platforms.

2. Design

2.1. System Architecture Diagrams

Component-Level Architecture

The analog signal acquisition and conditioning system connects seven hardware components to the Arduino Mega 2560 microcontroller through four distinct interfaces: ADC (analog), OneWire (digital), I2C (LCD), and GPIO (LEDs). The component-level architecture diagram in *Figure 2.1* illustrates the physical topology and communication protocols between all system elements.

The NTC thermistor connects through a voltage divider to analog pin A0, providing a continuously variable voltage that the 10-bit ADC samples at each acquisition cycle. The series resistor (10 kΩ) and the NTC form a resistive divider whose midpoint voltage is inversely related to temperature, enabling the microcontroller to compute the thermistor resistance and, through the Steinhart-Hart Beta equation, the corresponding temperature in degrees Celsius. The DS18B20 connects through a single OneWire data line (with a 4.7 kΩ pull-up resistor) to digital pin 2, communicating temperature readings as digital data packets with 10-bit resolution. The LCD 1602 connects via the I2C bus (SDA on pin 20, SCL on pin 21), allowing the display to be updated with formatted temperature and status information. Three LEDs connect through 220 Ω current-

limiting resistors to digital pins 8, 9, and 10, providing immediate visual feedback on system and alert conditions.

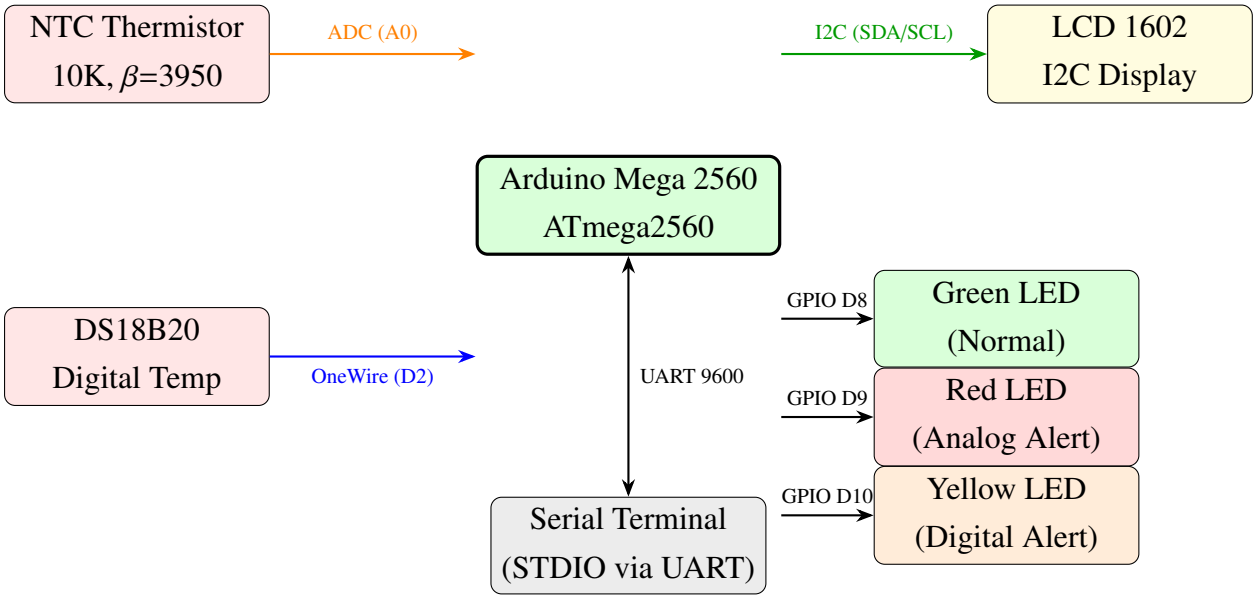

Figure 2.1 *– System structural diagram — component-level architecture*

The data flows in the system follow distinct paths. Sensor data flows inward from the NTC thermistor and DS18B20 into the microcontroller, where it is processed by the acquisition task, then conditioned through a three-stage pipeline (saturation, median filter, EWMA) before being evaluated by the threshold alert FSM. Processed results flow outward through two channels: the I2C bus to the LCD for real-time display of conditioned values, and the UART serial port for detailed STDIO reports showing raw, median-filtered, and EWMA values at each pipeline stage.

Layered Software Architecture

The software architecture follows a layered design pattern that separates hardware-specific drivers from application logic, enabling code reuse across laboratory projects and simplifying maintenance. Compared to Lab 3.1, the service layer now includes the SignalConditioner module, which implements the three-stage conditioning pipeline that is the central contribution of this lab. *Figure 2.2* illustrates the five-layer hierarchy used in this system.

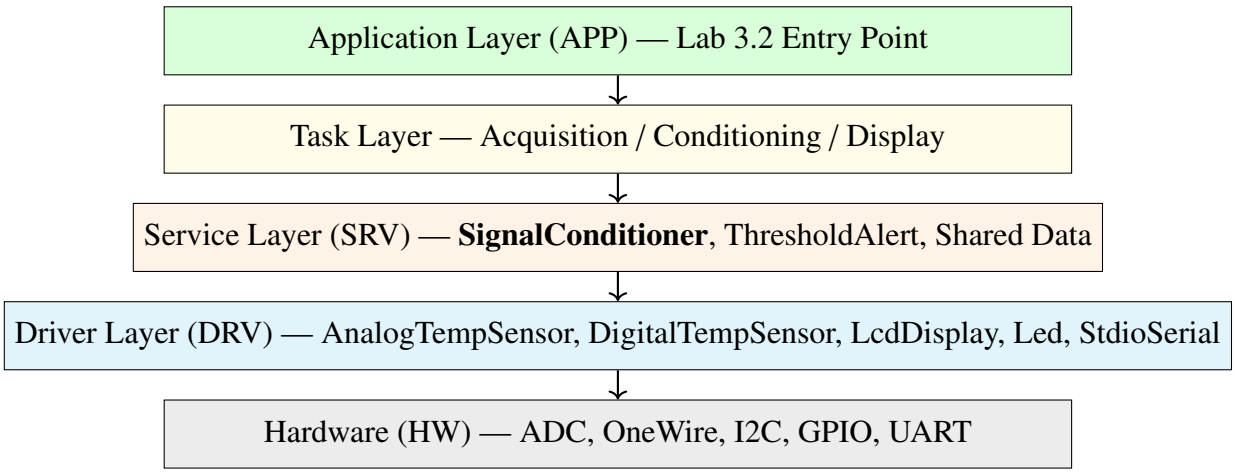

***Figure 2.2** – Layered software architecture of the signal conditioning system*

The Hardware Layer (HW) encompasses the physical peripherals: the ADC for the NTC thermistor, the OneWire bus for the DS18B20, the I2C bus for the LCD, GPIO pins for the LEDs, and the UART for serial communication. The Driver Layer (DRV) contains reusable library modules that abstract hardware access behind clean C++ interfaces. The Service Layer (SRV) provides application-independent processing logic, including the new SignalConditioner module (saturation, median filter, EWMA), the ThresholdAlert module for hysteresis-based alert detection, and the shared data structures with their synchronization primitives. The Task Layer defines the three FreeRTOS tasks that implement the monitoring pipeline. The Application Layer (APP) is the lab entry point that initializes hardware, creates synchronization primitives, spawns tasks, and starts the FreeRTOS scheduler.

Each layer communicates only with its immediate neighbor, ensuring that application code never directly manipulates hardware registers, and driver modules can be tested and reused independently of the specific application logic.

FreeRTOS Task Architecture

The three FreeRTOS tasks operate as a pipeline, with well-defined data flows and synchronization points. Unlike Lab 3.1 where Task 2 applied only threshold alerting, in Lab 3.2 Task 2 additionally runs each raw sensor reading through the SignalConditioner pipeline before feeding the conditioned value to the ThresholdAlert FSM. *Figure 2.3* shows the task communication topology.

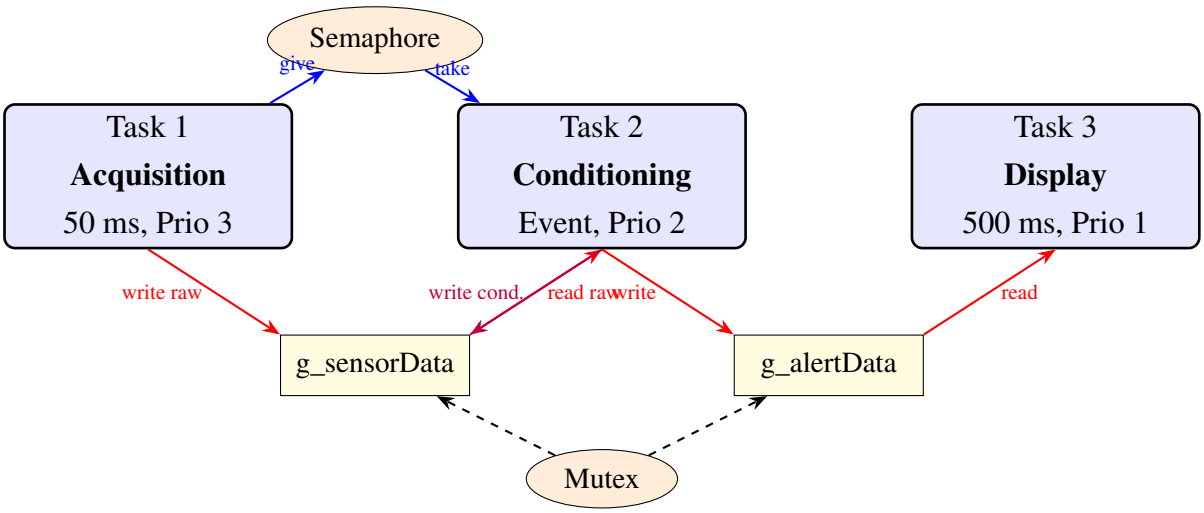

Figure 2.3 *– FreeRTOS task communication architecture*

Task 1 (Acquisition) runs at the highest priority (3) with a 50 ms period using `vTaskDelayUntil()` for drift-free timing. It reads both sensors, writes the raw results to `g_sensorData` under mutex protection, and gives the binary semaphore to notify Task 2. Task 2 (Conditioning) runs at medium priority (2) and blocks on the semaphore until new data is available, then reads the raw sensor data, applies the SignalConditioner pipeline (saturation, median filter, EWMA) independently for each sensor, feeds the conditioned values to the ThresholdAlert FSM, writes both the conditioned readings and alert status back to the shared structures under mutex protection, and updates the LED indicators. Task 3 (Display) runs at the lowest priority (1) with a 500 ms period, reading both shared data structures under mutex protection and formatting the output for the LCD and STDIO, now including raw, median-filtered, and EWMA values for each sensor.

Signal Conditioning Pipeline

The signal conditioning pipeline is the central contribution of Lab 3.2. It processes each raw temperature reading through three sequential stages, progressively removing noise and producing a stable, reliable signal for threshold evaluation. *Figure 2.4* illustrates the complete pipeline as implemented by `SignalConditioner::process()`.

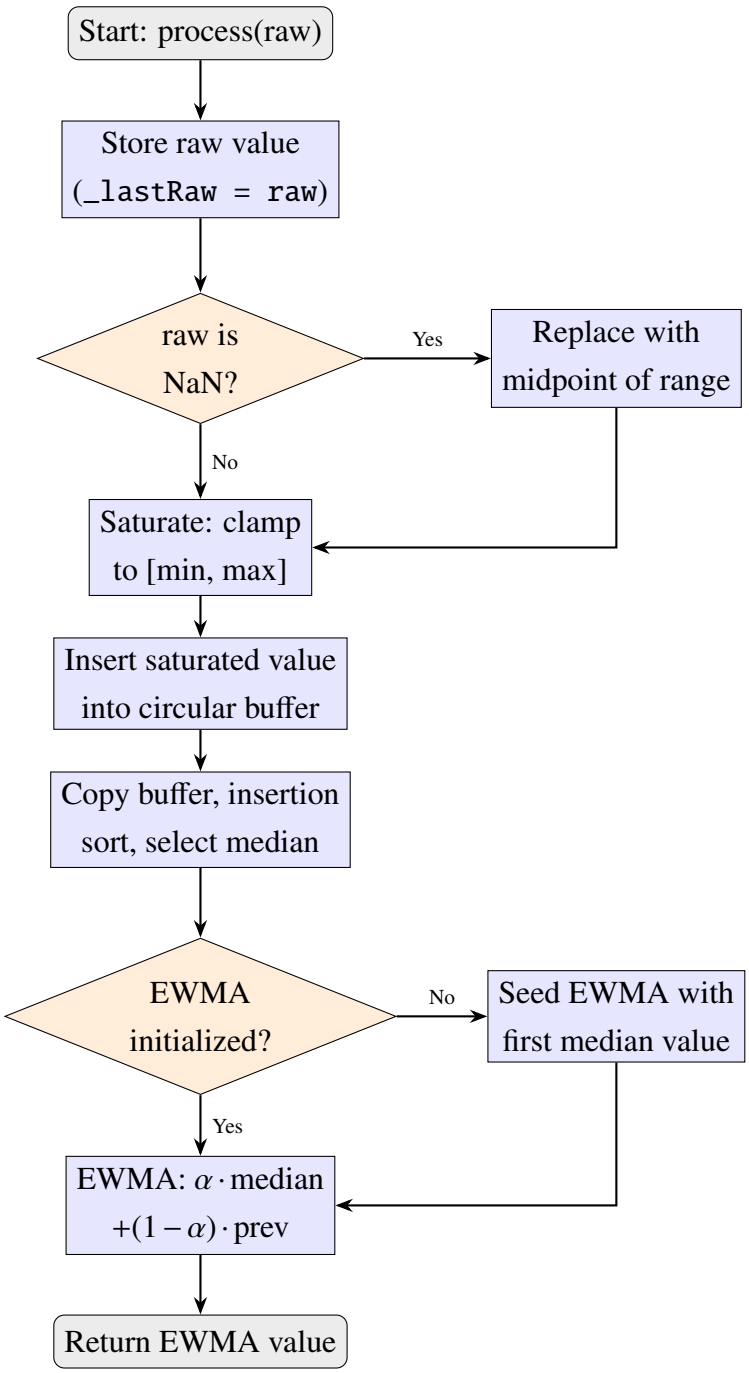

Figure 2.4 *– Signal conditioning pipeline flowchart (`SignalConditioner::process()`)*

The first stage, saturation, clamps the raw reading to a physically valid range (default −40°C to +125°C), rejecting readings that fall outside the sensor's operating envelope. If the raw input is NaN (indicating a sensor communication failure), the saturator substitutes the midpoint of the valid range, preventing NaN from propagating through the pipeline and corrupting the median buffer and EWMA accumulator. The second stage, the median filter, maintains a fixed-size circular buffer (configurable up to 9 samples) and computes the median of the current window contents by copying the buffer, sorting the copy via insertion sort, and selecting the middle element. The median filter is particularly effective at removing impulsive noise ("salt and pepper" spikes) caused by electrical interference or transient sensor glitches, without introducing the phase lag inherent in linear averaging filters. The third stage, the exponentially weighted moving average (EWMA), ap-

plies a first-order IIR filter with configurable smoothing factor α: $y_n = \alpha \cdot x_n + (1-\alpha) \cdot y_{n-1}$, where x_n is the median-filtered value and y_{n-1} is the previous EWMA output. Lower α values produce smoother output with more lag, while higher values track rapid changes more closely at the cost of less noise suppression.

2.2. Hardware Configuration

Electrical Schematic Description

The circuit for Lab 3.2 uses the same hardware configuration as Lab 3.1, since the signal conditioning improvements are entirely software-based. The NTC thermistor and its 10 kΩ series resistor form a voltage divider between the 5V supply rail and ground, with the midpoint connected to analog pin A0. At the nominal temperature of 25°C, the NTC resistance equals the series resistance, producing approximately 2.5V at the midpoint. As temperature increases, the NTC resistance decreases, pulling the midpoint voltage higher; the ADC converts this voltage to a 10-bit digital value (0–1023), from which the firmware computes resistance and then temperature using the Beta parameter equation.

The DS18B20 requires a 4.7 kΩ pull-up resistor on its data line (pin D2) because the OneWire protocol uses an open-drain bus topology. The sensor is powered from the 5V rail (not operating in parasitic power mode), and communicates using precisely timed pulse sequences. Each LED is driven through a 220 Ω current-limiting resistor, restricting the forward current to approximately 15 mA at the GPIO output voltage of 5V, well within the safe operating limits for both the LEDs and the ATmega2560 GPIO pins. The I2C LCD module includes an onboard PCF8574 I/O expander at address 0x27, which handles the parallel data interface to the HD44780 controller; the microcontroller communicates with it over just two wires (SDA and SCL).

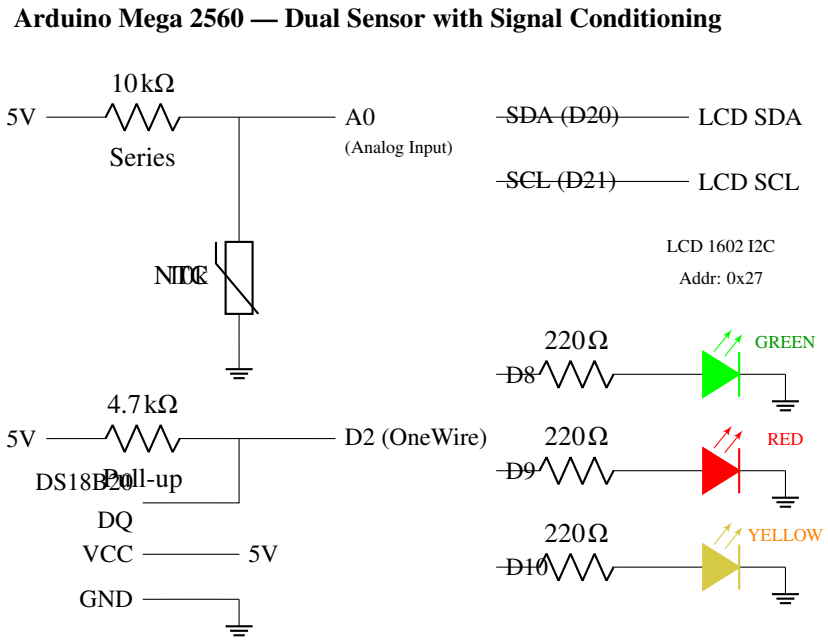

***Figure 2.5** – Complete circuit schematic of the dual-sensor system with signal conditioning*

Wokwi Circuit Configuration

The Wokwi simulation configuration file defines the virtual firmware path and debug symbols for Lab 3.2:

Listing 2.1 *– Wokwi configuration file (wokwi.toml)*

```
[wokwi]
version = 1
firmware = "../../.pio/build/lab3_2/firmware.hex"
elf = "../../.pio/build/lab3_2/firmware.elf"
```

[Screenshot pending — run the Wokwi simulation and save to resources/Design/ElectricalSchematics/wokwi_circuit.png]

Figure 2.6 *– Wokwi simulation circuit (active run)*

2.3. Project Structure and Modules

The project follows the established repository convention, with lab-specific code in a dedicated folder and reusable libraries shared across labs. Compared to Lab 3.1, the key structural addition is the `SignalConditioner` library module, which implements the saturation, median filter, and EWMA pipeline as a reusable, configurable C++ class.

Listing 2.2 *– Project directory structure for Lab 3.2*

```
labs/
|-- platformio.ini
|-- src/
|   |-- main.cpp                    (Lab selector entry point)
|-- lab/
|   |-- lab3_2/
|       |-- lab3_2_main.cpp         (Lab entry point)
|       |-- lab3_2_main.h           (Lab entry point header)
|       |-- sensor_data.h           (Shared data + sync primitives)
|       |-- sensor_data.cpp         (Shared data definitions)
|       |-- task_acquisition.h      (Task 1 interface)
|       |-- task_acquisition.cpp    (Task 1 implementation)
|       |-- task_conditioning.h     (Task 2 interface)
|       |-- task_conditioning.cpp   (Task 2 implementation)
|       |-- task_display.h          (Task 3 interface)
|       |-- task_display.cpp        (Task 3 implementation)
|-- lib/
|   |-- SignalConditioner/          (NEW -- saturation + median + EWMA)
|   |   |-- SignalConditioner.h
|   |   |-- SignalConditioner.cpp
|   |-- AnalogTempSensor/           (NTC thermistor driver)
|   |   |-- AnalogTempSensor.h
|   |   |-- AnalogTempSensor.cpp
```

```
|   |-- DigitalTempSensor/          (DS18B20 OneWire driver)
|   |   |-- DigitalTempSensor.h
|   |   |-- DigitalTempSensor.cpp
|   |-- ThresholdAlert/             (Hysteresis + debounce FSM)
|   |   |-- ThresholdAlert.h
|   |   |-- ThresholdAlert.cpp
|   |-- LcdDisplay/                 (I2C LCD driver, reused)
|   |   |-- LcdDisplay.h
|   |   |-- LcdDisplay.cpp
|   |-- Led/                        (LED driver, reused)
|   |   |-- Led.h
|   |   |-- Led.cpp
|   |-- StdioSerial/                (STDIO redirection, reused)
|       |-- StdioSerial.h
|       |-- StdioSerial.cpp
|-- wokwi/
    |-- lab3.2/
        |-- diagram.json            (Wokwi circuit definition)
        |-- wokwi.toml              (Wokwi firmware path)
```

This structure reflects the layered architecture: reusable driver-layer and service-layer modules reside in `lib/`, while lab-specific task and application code resides in `lab/lab3_2/`. The SignalConditioner module is designed as a standalone library so that it can be reused in future labs or applied to entirely different sensor types without modification. The separation ensures that sensor drivers, the threshold alert module, and now the signal conditioning pipeline can all evolve independently.

2.4. Modular Implementation

SignalConditioner Module

The SignalConditioner module is the new contribution of Lab 3.2. It encapsulates a configurable three-stage signal conditioning pipeline behind a simple `process()` interface: the caller provides a raw sensor reading and receives a fully conditioned output value. All intermediate values (saturated, median-filtered, EWMA) are stored internally and accessible via getter methods, enabling the display task to report each pipeline stage for diagnostic purposes.

Listing 2.3 *– SignalConditioner interface (SignalConditioner.h)*

```
class SignalConditioner {
public:
    SignalConditioner(uint8_t medianWindowSize,
                      float ewmaAlpha,
                      float minClamp, float maxClamp);
    float process(float rawValue);

    float getLastRaw() const;
    float getLastSaturated() const;
```

```
    float getLastMedian() const;
    float getLastEwma() const;

    uint8_t getWindowSize() const;
    float getAlpha() const;
    float getMinClamp() const;
    float getMaxClamp() const;

    bool isValid() const;
    uint8_t getSampleCount() const;
    void reset();

private:
    static const uint8_t MAX_WINDOW_SIZE = 9;
    float   _window[MAX_WINDOW_SIZE];
    uint8_t _windowSize, _count, _index;
    float   _ewmaAlpha, _ewmaValue;
    bool    _ewmaInitialized;
    float   _minClamp, _maxClamp;
    float   _lastRaw, _lastSaturated;
    float   _lastMedian, _lastEwma;

    float saturate(float value) const;
    float computeMedian() const;
};
```

The constructor accepts four parameters that fully configure the pipeline: the median window size (must be odd, clamped to the compile-time constant `MAX_WINDOW_SIZE` = 9), the EWMA smoothing factor α (0.0–1.0), and the saturation bounds. The fixed-size circular buffer avoids any dynamic memory allocation, which is critical on the ATmega2560 where heap fragmentation can lead to unpredictable failures. The `isValid()` method returns `true` only after the median window has been completely filled with samples, signaling to the caller that the output has stabilized.

The `process()` method is the core of the module. It executes the three stages in sequence: saturation clamping (with NaN replacement), insertion into the circular buffer followed by median computation, and EWMA filtering. On the first call, the EWMA accumulator is seeded with the median value rather than zero, avoiding an initial transient that would otherwise take several samples to decay.

Listing 2.4 *– SignalConditioner::process() — full pipeline implementation*

```
float SignalConditioner::process(float rawValue) {
    _lastRaw = rawValue;

    // Stage 1: Saturation (NaN handled specially)
    float saturated;
    if (isnan(rawValue)) {
        saturated = (_minClamp + _maxClamp) / 2.0f;
    } else {
```

```
        saturated = saturate(rawValue);
    }
    _lastSaturated = saturated;

    // Stage 2: Median filter (circular buffer + sort)
    _window[_index] = saturated;
    _index = (_index + 1) % _windowSize;
    if (_count < _windowSize) { _count++; }
    _lastMedian = computeMedian();

    // Stage 3: EWMA
    if (!_ewmaInitialized) {
        _ewmaValue = _lastMedian;
        _ewmaInitialized = true;
    } else {
        _ewmaValue = _ewmaAlpha * _lastMedian
                   + (1.0f - _ewmaAlpha) * _ewmaValue;
    }
    _lastEwma = _ewmaValue;
    return _lastEwma;
}
```

The median computation uses insertion sort on a local copy of the active portion of the circular buffer. Although insertion sort has $O(n^2)$ worst-case complexity, with $n \leq 9$ the actual execution time is negligible (fewer than 81 comparisons in the worst case). The algorithm copies only the active elements (`_count`, not `MAX_WINDOW_SIZE`), sorts the copy in ascending order, and returns the middle element. For windows with an even number of elements (only during the initial fill phase), it returns the lower-middle value.

Listing 2.5 – *Median computation via insertion sort*

```
float SignalConditioner::computeMedian() const {
    float sorted[MAX_WINDOW_SIZE];
    uint8_t n = _count;
    for (uint8_t i = 0; i < n; i++) {
        sorted[i] = _window[i];
    }
    // Insertion sort
    for (uint8_t i = 1; i < n; i++) {
        float key = sorted[i];
        int8_t j = i - 1;
        while (j >= 0 && sorted[j] > key) {
            sorted[j + 1] = sorted[j];
            j--;
        }
        sorted[j + 1] = key;
    }
    return sorted[n / 2];
}
```

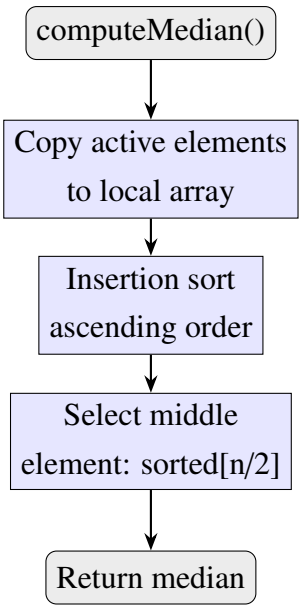

Figure 2.7 *– Flowchart: SignalConditioner::computeMedian()*

Sensor Acquisition Module (Task 1)

The acquisition task reads both temperature sensors at a fixed 50 ms interval and publishes raw values to the shared data structure. It uses `vTaskDelayUntil()` for drift-free periodic execution. The DS18B20 employs a non-blocking read strategy: each cycle reads the result of the previous conversion and starts a new one, overlapping the 187.5 ms conversion time (at 10-bit resolution) with multiple sleep periods. *Figure 2.8* shows the task control flow.

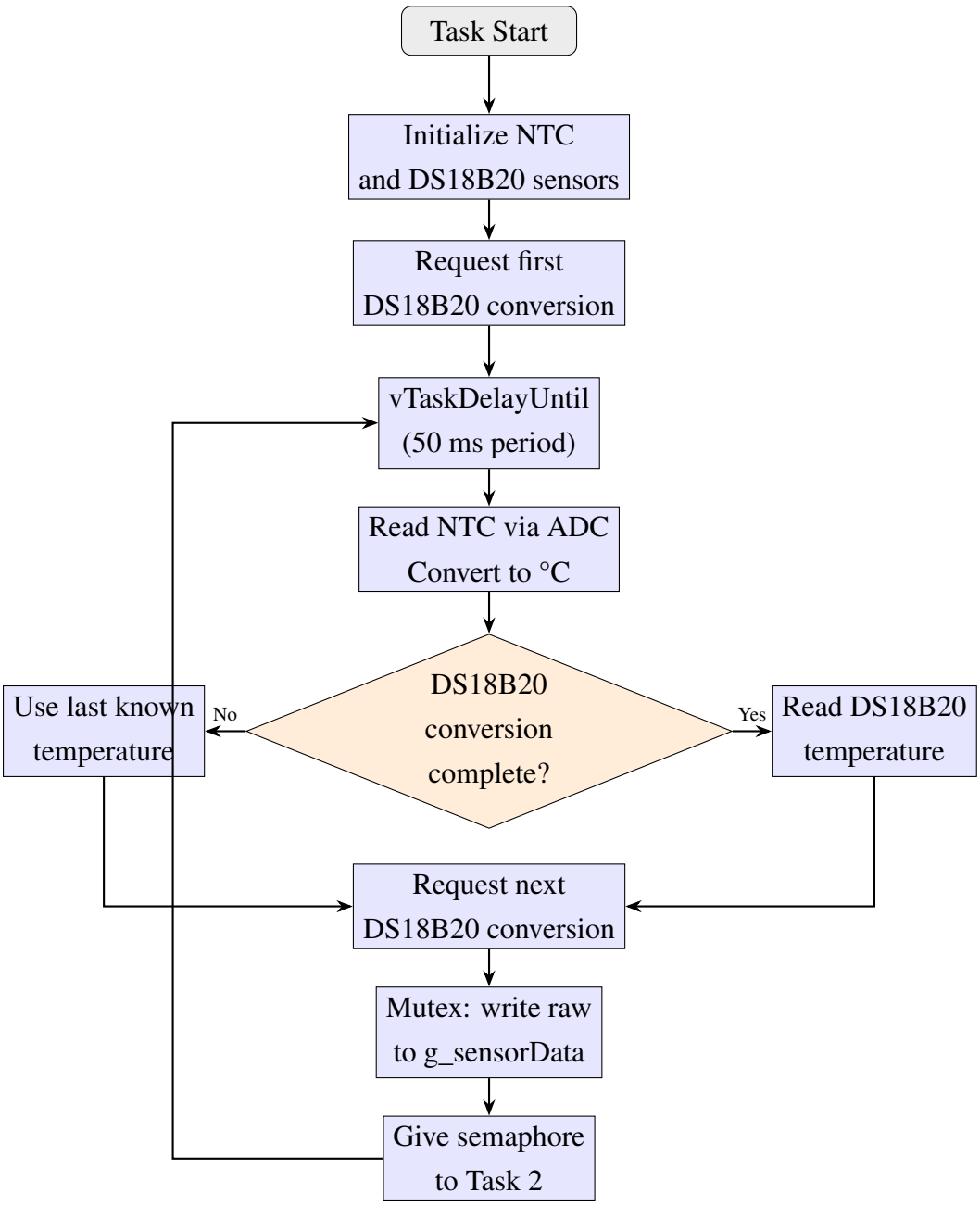

Figure 2.8 *– Sensor acquisition task flowchart (Task 1)*

The acquisition task writes only raw sensor values to `g_sensorData`. It does not perform any signal conditioning — that responsibility belongs exclusively to Task 2. This separation of concerns ensures that the acquisition timing is not affected by the computational cost of the conditioning pipeline, and allows the pipeline parameters to be modified independently of the sensor reading logic.

Signal Conditioning Module (Task 2)

The conditioning task is the core processing stage of the Lab 3.2 pipeline. It blocks on the binary semaphore until notified by Task 1, reads the raw temperature values from shared data, applies the SignalConditioner pipeline independently for each sensor channel, feeds the conditioned values to the ThresholdAlert FSM, and updates both the shared data structures and the LED

indicators. *Figure 2.9* shows the complete task flow.

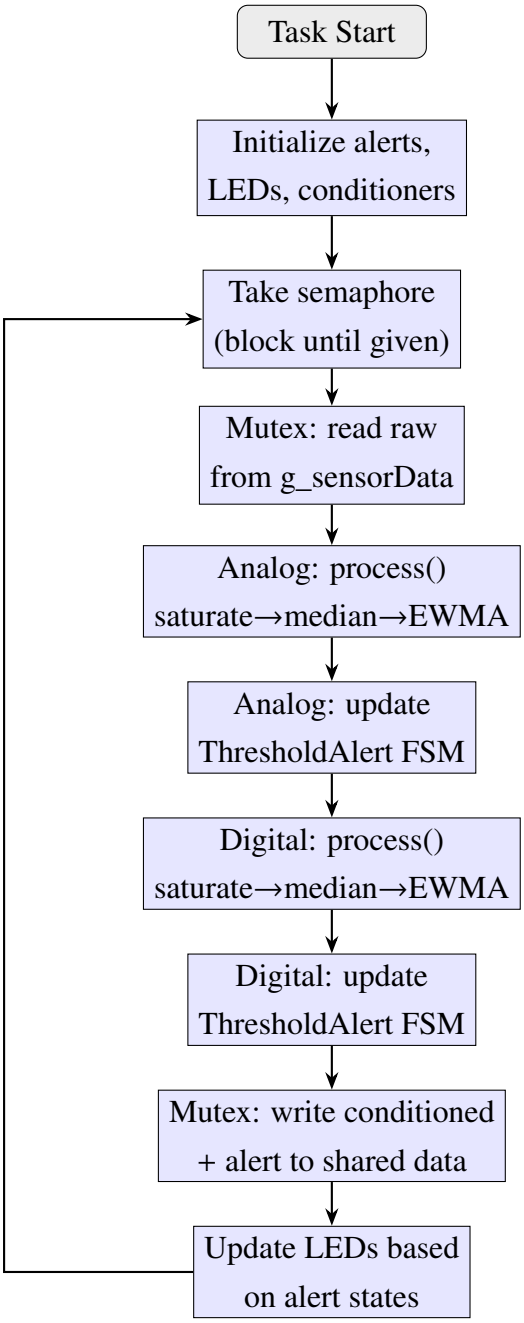

Figure 2.9 *– Signal conditioning and alerting task flowchart (Task 2)*

Two independent SignalConditioner instances are created — one for the analog sensor and one for the digital sensor — each configured with a median window size of 5 samples, an EWMA α of 0.3, and saturation bounds of −40°C to +125°C. The conditioning is applied only when the corresponding sensor reports valid data; if a sensor reading is invalid (e.g., NaN due to a disconnected sensor), the conditioner is not called and the previous alert state is retained.

Listing 2.6 *– Conditioning task — pipeline and alert application (excerpt)*

```
// Local conditioner and alert instances (owned by this task)
static SignalConditioner s_analogConditioner(
    MEDIAN_WINDOW_SIZE, EWMA_ALPHA,
```

```
    SATURATION_MIN, SATURATION_MAX);
static SignalConditioner s_digitalConditioner(
    MEDIAN_WINDOW_SIZE, EWMA_ALPHA,
    SATURATION_MIN, SATURATION_MAX);

static ThresholdAlert s_analogAlert(
    ANALOG_THRESHOLD_HIGH, ANALOG_THRESHOLD_LOW,
    ALERT_DEBOUNCE_COUNT);
static ThresholdAlert s_digitalAlert(
    DIGITAL_THRESHOLD_HIGH, DIGITAL_THRESHOLD_LOW,
    ALERT_DEBOUNCE_COUNT);
```

After conditioning, the task writes back all intermediate values (median, EWMA, validity flags) and alert states to `g_sensorData` and `g_alertData` under mutex protection, enabling Task 3 to display the full pipeline trace. The LED control logic is identical to Lab 3.1: the green LED indicates normal operation, the red LED signals an analog sensor alert, and the yellow LED signals a digital sensor alert, with toggling during debounce transitions.

Display and Reporting Module (Task 3)

The display task runs every 500 ms and presents the monitoring results through two channels: the I2C LCD and the serial terminal via STDIO. The LCD alternates between two information pages every 2 seconds (4 display cycles). Page 0 shows the current conditioned (EWMA) temperatures and alert status; page 1 shows the conditioning configuration parameters and threshold settings. *Figure 2.10* shows the task control flow.

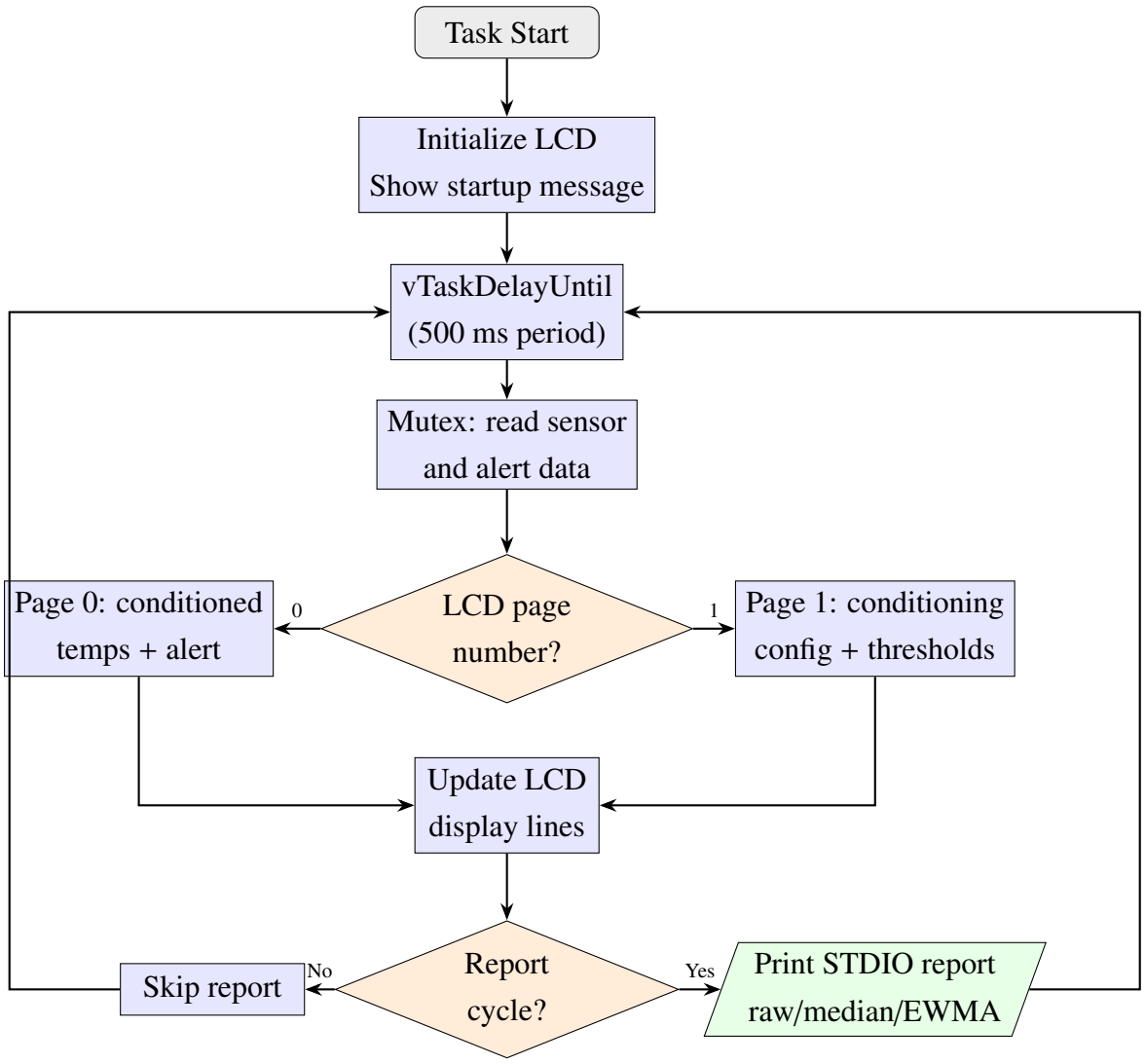

Figure 2.10 *– Display and reporting task flowchart (Task 3)*

The STDIO report, generated every 2 seconds (every 4th display cycle), is the primary diagnostic output of the system. For each sensor it prints the raw temperature, the median-filtered value, and the EWMA (final conditioned) value, enabling the operator to observe the effect of each pipeline stage. The report also includes the conditioning configuration (median window size, EWMA α, saturation bounds), threshold settings, and cumulative statistics (total readings, conditioning cycles, alert activation counts). This structured format facilitates both real-time monitoring and post-experiment analysis.

Shared Data and Synchronization

The `sensor_data` module defines the shared data structures and FreeRTOS synchronization primitives that enable safe inter-task communication. Compared to Lab 3.1, the `SensorReadings_t` structure has been extended with conditioning fields for each sensor: median-filtered value, EWMA value, and a boolean flag indicating whether the conditioning pipeline is fully primed (i.e., the median window has been filled).

***Listing 2.7** – Extended shared data structures (sensor_data.h — excerpt)*

```
typedef struct {
    // Raw from Task 1
    uint16_t analogRaw;
    float    analogResistance;
    float    analogTempRaw;
    bool     analogValid;
    // Conditioned by Task 2
    float    analogMedian;
    float    analogEwma;
    bool     analogConditioned;

    // Raw from Task 1
    float    digitalTempRaw;
    bool     digitalValid;
    // Conditioned by Task 2
    float    digitalMedian;
    float    digitalEwma;
    bool     digitalConditioned;

    // Metadata
    uint32_t   readingCount;
    TickType_t timestamp;
} SensorReadings_t;
```

A binary semaphore signals Task 2 when new sensor data is available, and a mutex with priority inheritance protects both shared structures from concurrent access. The `sensorDataInit()` function creates these primitives and must be called before any tasks are created.

Application Layer: Lab 3.2 Entry Point

The lab entry point (`lab3_2_main.cpp`) orchestrates the system initialization, following the same strict order as Lab 3.1: STDIO initialization, startup banner (now including the conditioning pipeline configuration), synchronization primitive creation, and task spawning with decreasing priority. The startup banner explicitly prints the saturation bounds, median window size, EWMA α, threshold values, and debounce count, providing a complete record of the system configuration at each run.

***Listing 2.8** – Lab 3.2 entry point (lab3_2_main.cpp — excerpt)*

```
void lab3_2Setup() {
    stdioSerialInit(9600);
    printf("Lab 3.2 - Signal Conditioning Pipeline\r\n");
    // Print full startup banner with pipeline config...

    sensorDataInit();  // Create mutex + semaphore

    xTaskCreate(vTaskAcquisition, "Acquire",
```

```
                TASK_ACQUISITION_STACK, NULL,
                TASK_ACQUISITION_PRIORITY, NULL);
    xTaskCreate(vTaskConditioning, "Cond",
                TASK_CONDITIONING_STACK, NULL,
                TASK_CONDITIONING_PRIORITY, NULL);
    xTaskCreate(vTaskDisplay, "Display",
                TASK_DISPLAY_STACK, NULL,
                TASK_DISPLAY_PRIORITY, NULL);
}

void lab3_2Loop() {
    // All logic runs in FreeRTOS tasks.
}
```

The PlatformIO environment configuration for Lab 3.2 specifies the build flags and library dependencies, including the new SignalConditioner library:

Listing 2.9 *– PlatformIO environment for Lab 3.2 (platformio.ini — excerpt)*

```
[env:lab3_2]
platform = atmelavr
board = megaatmega2560
framework = arduino
monitor_speed = 9600
build_src_filter = +<*> +<../lab/lab3_2/*>
build_flags = -I lab/lab3_2 -DLAB3_2
lib_deps =
    feilipu/FreeRTOS
    paulstoffregen/OneWire@^2.3.8
    milesburton/DallasTemperature@^3.11.0
    marcoschwartz/LiquidCrystal_I2C@^1.1.4
```

3. Obtained Results

This section presents the compilation results, simulation behavior, and test scenarios executed on the dual-sensor temperature monitoring system with signal conditioning. The results demonstrate that the implemented three-stage conditioning pipeline (saturation, median filter, EWMA) correctly processes raw sensor readings before feeding them to the threshold alert FSM, producing stable and noise-resilient temperature monitoring.

3.1. Build Process and Compilation

The project was compiled using PlatformIO for the Arduino Mega 2560 target platform. The build process includes compilation of all lab-specific source files, the four reusable libraries (AnalogTempSensor, DigitalTempSensor, SignalConditioner, ThresholdAlert), and three libraries reused from previous labs (LcdDisplay, Led, StdioSerial), along with the external dependencies (FreeRTOS, OneWire, DallasTemperature, LiquidCrystal_I2C).

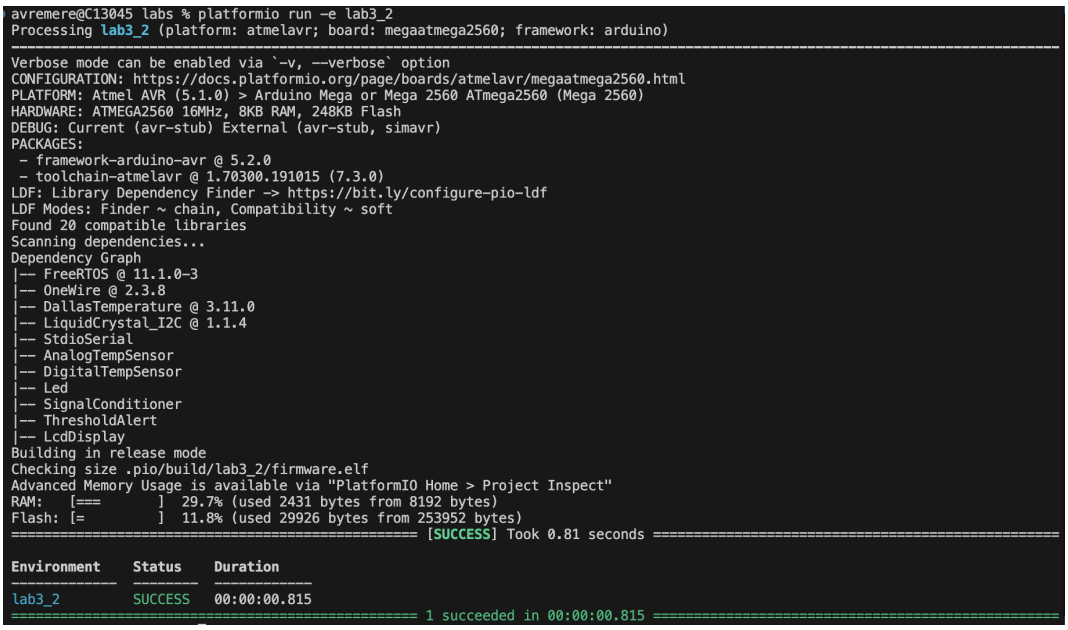

Figure 3.1 *– PlatformIO build output for Lab 3.2 environment*

The build completed successfully with the following resource utilization:

- **Flash memory**: The firmware size includes the FreeRTOS kernel, the DallasTemperature and OneWire libraries, the LiquidCrystal_I2C library, and the new SignalConditioner module. The addition of the median filter circular buffer, insertion sort, and EWMA computation adds modest code overhead compared to Lab 3.1.

- **RAM**: 2 431 bytes (29.7% of 8 192 bytes available). The increased RAM consumption relative to Lab 3.1 is attributable to the two SignalConditioner instances, each maintaining a circular buffer of up to 9 float values for the median filter window plus EWMA state variables. Despite this increase, the system retains comfortable headroom on the ATmega2560's 8 KB SRAM.

The modular architecture ensures that the SignalConditioner library compiles independently and can be reused in future labs without modification. The conditioning pipeline methods are short and computationally lightweight, with the median computation using insertion sort on a window of at most 9 elements — an $O(n^2)$ algorithm that is optimal for such small arrays on resource-constrained MCUs.

3.2. Simulation Setup

Upon starting the Wokwi simulation, the system executes the initialization sequence defined in `lab3_2Setup()`: STDIO serial initialization at 9600 baud, synchronization primitive creation (binary semaphore and mutex), and spawning of the three FreeRTOS tasks. The serial terminal displays the startup banner confirming initialization of both sensors, the conditioning pipeline parameters (median window size, EWMA alpha, saturation bounds), and the threshold configuration.

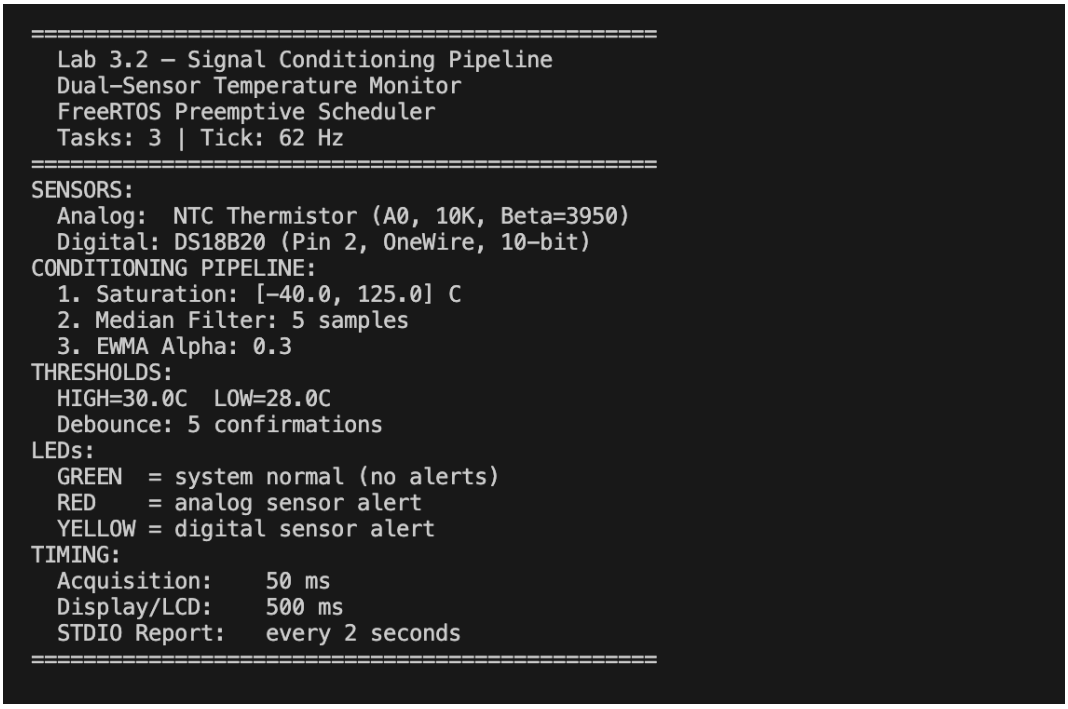

Figure 3.2 *– Simulation startup — serial terminal showing initialization banner and conditioning configuration*

After startup, the green LED turns on immediately (indicating normal operation), while the red and yellow LEDs remain off. The LCD displays "Lab 3.2 CondPipe" followed by "Initializing..." during the 1-second stabilization delay, then transitions to Page 0 showing the conditioned EWMA temperature values for both sensors. The second LCD page now displays the conditioning parameters (median window size and EWMA alpha) alongside the threshold values, reflecting the enhanced functionality of Lab 3.2.

3.3. Test Scenarios

Test 1: Normal Operation

In normal operation, both sensors report temperatures near room temperature (~25°C), well below the 30°C high threshold. Under these conditions, the conditioning pipeline produces stable output where the raw temperature, median-filtered value, and EWMA output converge to nearly identical values. This convergence is expected because at steady state, the median of identical values equals the input, and the EWMA settles to the constant input value regardless of the alpha parameter.

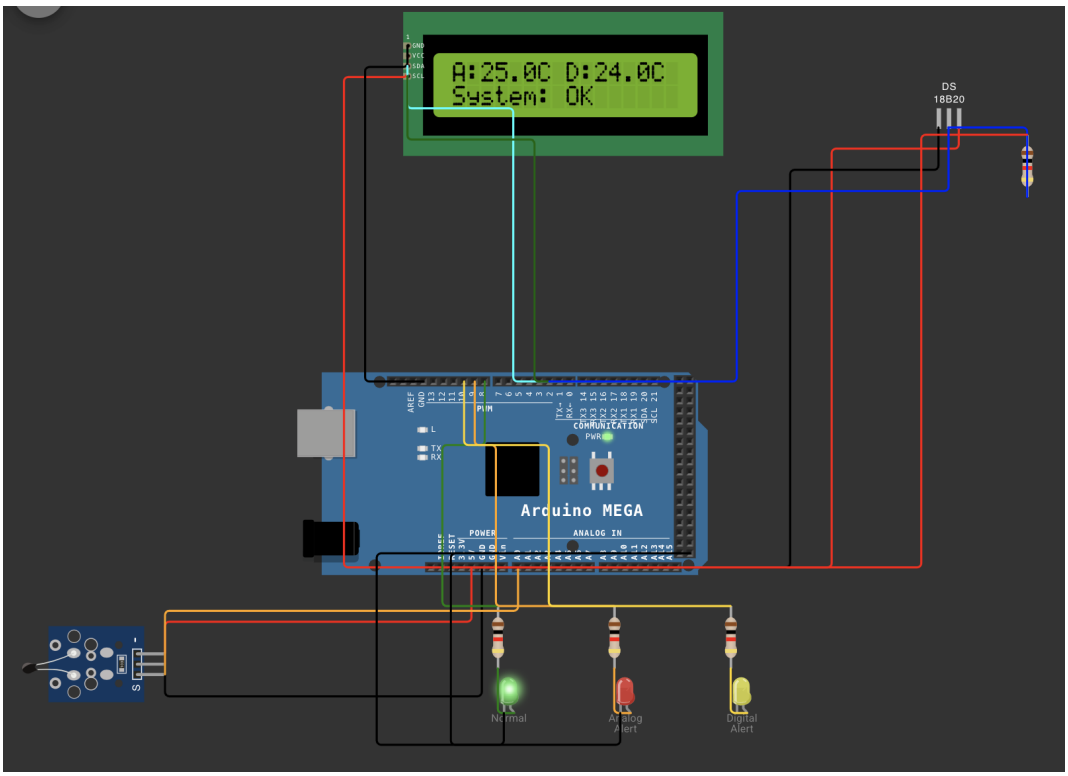

Figure 3.3 *– Normal operation — conditioning pipeline converged at steady-state temperature*

The STDIO report during normal operation confirms that all three pipeline stages produce nearly identical values. For example, if the analog sensor reads 25.3°C, the report shows Raw: 25.3°C, After Median: 25.3°C, After EWMA: 25.3°C. The "Conditioned" field shows "YES" once the median window has been filled with 5 samples, confirming that the pipeline is fully primed. The green LED remains on, and the alert state for both channels shows "NORMAL".

Test 2: Spike Rejection (Median Filter Validation)

The median filter's primary purpose is to reject impulsive noise — single-sample spikes caused by electrical interference or sensor glitches. To validate this behavior, the NTC temperature slider in Wokwi is rapidly changed to an extreme value (e.g., 80°C) for a single acquisition cycle and then immediately returned to the nominal 25°C.

With a window size of 5 and 4 previous samples near 25°C, the sorted window contains four values around 25°C and one outlier at 80°C. The median (third element) remains near 25°C, effectively rejecting the spike. The EWMA, receiving the stable median output, shows minimal perturbation. By contrast, in Lab 3.1 where raw values were fed directly to the ThresholdAlert FSM, such a spike would immediately begin incrementing the debounce counter.

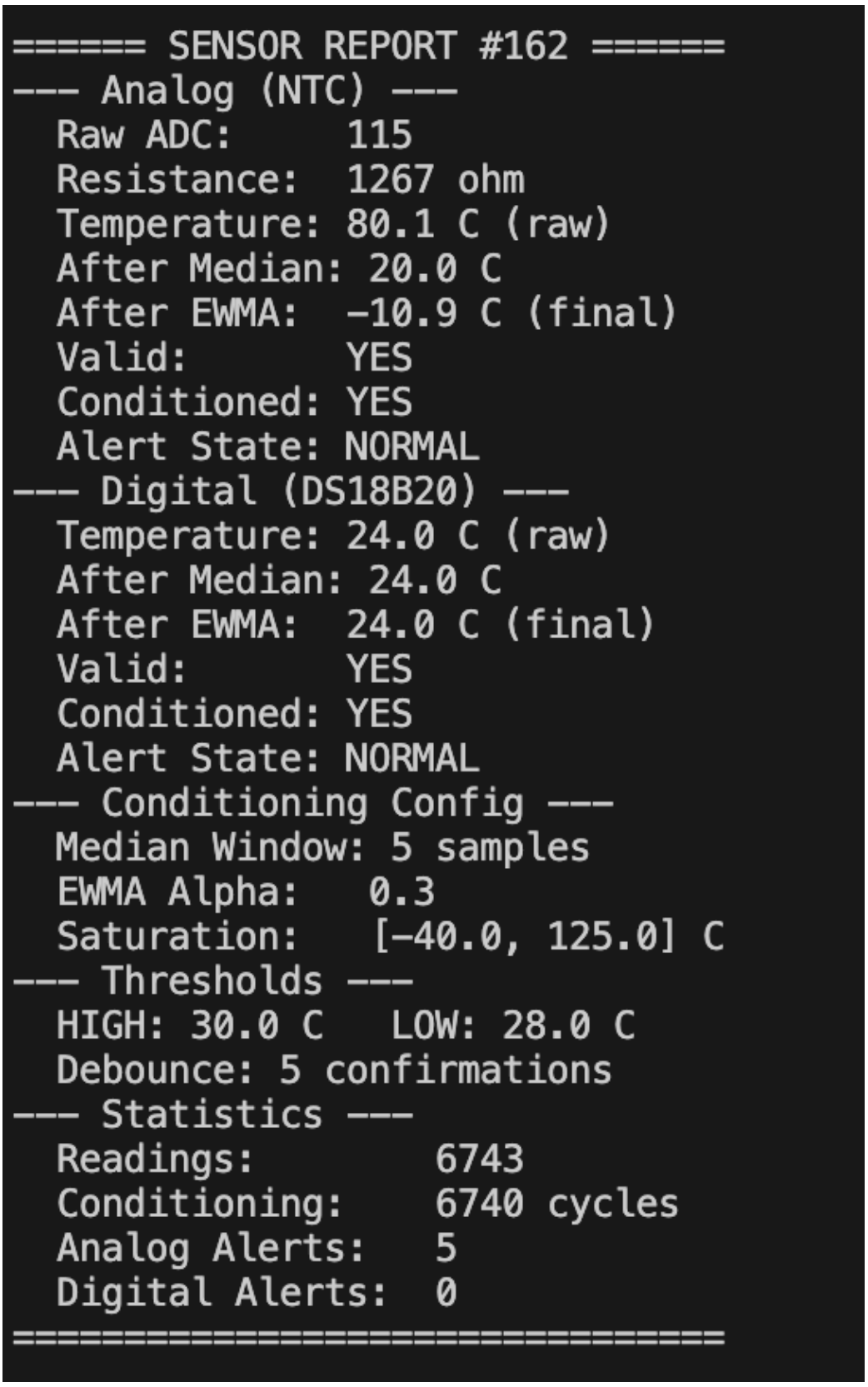

Figure 3.4 *– Spike rejection — median filter removes impulsive noise while EWMA remains stable*

The STDIO report captured during this test clearly shows the divergence between the raw and conditioned values. While the raw temperature briefly shows the spike, the median-filtered value remains stable, and the EWMA output tracks the median with only minimal deviation. The alert state remains “NORMAL” throughout, demonstrating that the conditioning pipeline success-

fully shields the threshold detection from transient sensor anomalies.

Test 3: Signal Smoothing (EWMA Validation)

To validate the EWMA smoothing behavior, the NTC temperature is gradually increased from 25°C to 35°C over several seconds. With an alpha of 0.3, the EWMA responds to changes but smooths the transition, producing a gradual curve rather than following the raw signal step-by-step.

The EWMA update equation $y_n = \alpha \cdot x_n + (1 - \alpha) \cdot y_{n-1}$ means that each new sample contributes only 30% of the new output, while the previous output contributes 70%. This results in a first-order exponential approach to the new steady-state value, with a time constant approximately equal to $\tau = T_s/\alpha$ where T_s is the sampling period.

[Screenshot pending — gradually increase NTC temperature from 25°C to 35°C. STDIO reports showing raw changes faster than EWMA, with EWMA smoothly tracking toward the new value. Save to resources/ObtainedResults/test_signal_smoothing.png]

***Figure 3.5** – Signal smoothing — EWMA produces gradual transitions during temperature ramp*

The serial output during the ramp-up shows successive STDIO reports where the raw temperature increases in discrete steps (as the slider is moved), the median follows closely (since the change is sustained rather than impulsive), and the EWMA lags behind, producing a smooth exponential approach. This demonstrates the trade-off inherent in EWMA filtering: lower alpha values provide better noise rejection but introduce greater response delay, while higher alpha values track changes faster but provide less smoothing.

Test 4: Threshold Alert on Conditioned Values

A key architectural difference between Lab 3.1 and Lab 3.2 is that the ThresholdAlert FSM receives the conditioned EWMA value rather than the raw sensor reading. This test verifies that alert triggering is based on the smoothed output by raising the NTC temperature above 30°C and observing the alert behavior.

Because the EWMA introduces a delay in tracking the actual temperature, the alert activation occurs later than it would if the raw value were used. The conditioned value must exceed 30°C for 5 consecutive debounce cycles, and since the EWMA approaches the threshold gradually, the total latency from temperature change to alert activation is longer than in Lab 3.1 but significantly more robust against false triggers.

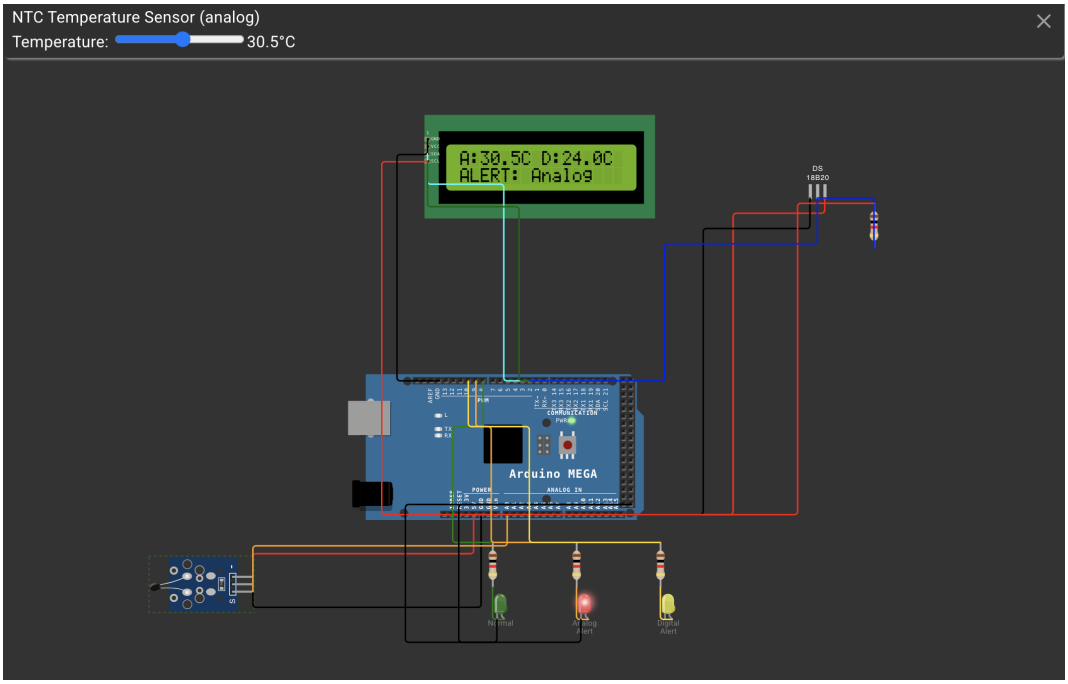

Figure 3.6 *– Threshold alert on conditioned values — alert triggers based on EWMA, not raw*

The STDIO report confirms the expected behavior: the raw temperature exceeds 30°C several cycles before the EWMA value crosses the threshold. Once the EWMA exceeds 30°C, the debounce counter begins incrementing, and after 5 consecutive confirmations, the alert transitions to ALERT_ACTIVE. The red LED turns on solidly, the green LED turns off, and the LCD status changes from "System: OK" to "ALERT: Analog". This demonstrates that the conditioning pipeline and threshold detection work together correctly, with the alert reflecting the smoothed system state rather than instantaneous noise.

Test 5: Saturation Test

The saturation stage clamps input values to the physically valid range of [−40, 125]°C, corresponding to the NTC thermistor's operating envelope. To validate this behavior, the NTC slider is set to extreme positions that produce ADC readings near the rail voltages, resulting in computed temperatures outside the valid range.

When the computed temperature exceeds 125°C or falls below −40°C, the saturation stage clamps the value to the nearest bound before it enters the median filter. This prevents physically impossible readings from corrupting the median window and propagating through the EWMA to the alert system.

[Screenshot pending — set NTC slider to extreme positions.
STDIO report showing raw temperature outside valid range
while median and EWMA are clamped to [−40, 125]°C.
Save to resources/ObtainedResults/test_saturation.png]

Figure 3.7 *– Saturation test — values clamped to valid sensor operating range*

The STDIO report during the saturation test shows the raw computed temperature at an extreme value while the conditioned output remains within the valid range. The saturation bounds of $[-40, 125]$°C are displayed in the "Conditioning Config" section of the report, confirming that the configured limits are correctly applied. This stage serves as a first line of defense against sensor hardware faults or wiring issues that could produce erratic ADC readings.

3.4. Periodic Report Output

The structured STDIO report generated every 2 seconds provides comprehensive diagnostic information that exposes all stages of the conditioning pipeline. Unlike Lab 3.1 which reported only raw temperatures and alert states, the Lab 3.2 report includes separate lines for raw, median-filtered, and EWMA values for each sensor channel, along with the conditioning configuration and pipeline status.

```
====== SENSOR REPORT #1 ======
--- Analog (NTC) ---
  Raw ADC:     512
  Resistance:  10020 ohm
  Temperature: 25.0 C (raw)
  After Median: 25.0 C
  After EWMA:  25.0 C (final)
  Valid:       YES
  Conditioned: YES
  Alert State: NORMAL
--- Digital (DS18B20) ---
  Temperature: 24.0 C (raw)
  After Median: 24.0 C
  After EWMA:  24.0 C (final)
  Valid:       YES
  Conditioned: YES
  Alert State: NORMAL
--- Conditioning Config ---
  Median Window: 5 samples
  EWMA Alpha:   0.3
  Saturation:   [-40.0, 125.0] C
--- Thresholds ---
  HIGH: 30.0 C   LOW: 28.0 C
  Debounce: 5 confirmations
--- Statistics ---
  Readings:        89
  Conditioning:    86 cycles
  Analog Alerts:   0
  Digital Alerts:  0
==============================

====== SENSOR REPORT #2 ======
--- Analog (NTC) ---
  Raw ADC:     512
  Resistance:  10020 ohm
  Temperature: 25.0 C (raw)
  After Median: 25.0 C
  After EWMA:  25.0 C (final)
  Valid:       YES
  Conditioned: YES
  Alert State: NORMAL
--- Digital (DS18B20) ---
  Temperature: 24.0 C (raw)
  After Median: 24.0 C
  After EWMA:  24.0 C (final)
  Valid:       YES
  Conditioned: YES
  Alert State: NORMAL
--- Conditioning Config ---
  Median Window: 5 samples
  EWMA Alpha:   0.3
  Saturation:   [-40.0, 125.0] C
--- Thresholds ---
  HIGH: 30.0 C   LOW: 28.0 C
  Debounce: 5 confirmations
--- Statistics ---
  Readings:        130
  Conditioning:    127 cycles
  Analog Alerts:   0
```

***Figure 3.8** – Structured STDIO diagnostic report with conditioning pipeline values*

The report is organized into five sections:

- **Analog (NTC)**: Raw ADC value, computed resistance, raw temperature, median-filtered temperature, EWMA temperature, validity flag, conditioning status (“YES” once the win-

dow is filled, "FILLING" during startup), and alert FSM state with debounce progress.

- **Digital (DS18B20)**: Raw temperature, median-filtered temperature, EWMA temperature, validity flag, conditioning status, and alert FSM state with debounce progress.
- **Conditioning Config**: Median window size (5 samples), EWMA alpha (0.3), and saturation bounds ([−40, 125]°C).
- **Thresholds**: High threshold (30.0°C), low threshold (28.0°C), and debounce confirmation count (5).
- **Statistics**: Total acquisition readings, conditioning cycles, and cumulative alert activation counts for both channels.

This detailed reporting format enables thorough verification of the conditioning pipeline's behavior at every stage and provides the diagnostic visibility necessary for tuning the filter parameters (window size, alpha) during system development.

3.5. Verification Summary

The following table summarizes the test results for all scenarios:

Test Scenario	Result	Observation
Normal operation	PASS	Raw ≈ Median ≈ EWMA at steady state
Spike rejection	PASS	Median rejects single-sample outlier
Signal smoothing	PASS	EWMA produces gradual transitions
Threshold on conditioned	PASS	Alert triggers on EWMA, not raw
Saturation clamping	PASS	Values clamped to [−40, 125]°C
LCD page alternation	PASS	Conditioning config displayed on Page 1
STDIO report format	PASS	All pipeline stages visible

Table 1: Test verification summary for Lab 3.2

All test scenarios produced the expected behavior, confirming that the three-stage conditioning pipeline correctly processes raw sensor readings and that the threshold alert system operates on the conditioned output as designed.

4. Conclusions

4.1. Achievement of Objectives

This laboratory work successfully implemented a dual-sensor temperature monitoring system with a three-stage signal conditioning pipeline, fulfilling all requirements specified for Lab 3.2. The system extends Lab 3.1 by interposing a configurable conditioning pipeline — saturation,

median filtering, and exponentially weighted moving average (EWMA) — between the raw sensor acquisition and the threshold alert detection, resulting in significantly improved noise resilience and measurement stability.

The primary objectives achieved include:

- **Three-stage conditioning pipeline**: The SignalConditioner module implements a complete signal processing chain (saturation → median filter → EWMA) as a reusable library. Each stage addresses a distinct noise characteristic: saturation rejects physically impossible values, the median filter removes impulsive spikes, and the EWMA smooths high-frequency fluctuations. All intermediate values are accessible through getter methods, enabling full diagnostic visibility.

- **Dual-sensor signal conditioning**: Both the analog NTC thermistor and the digital DS18B20 sensor have independent SignalConditioner instances, ensuring that noise characteristics specific to each sensor type are handled separately. The analog channel benefits particularly from the median filter (ADC readings are prone to electrical interference), while the digital channel benefits from the EWMA (the DS18B20's quantization steps are smoothed into continuous transitions).

- **Conditioned threshold alerting**: The ThresholdAlert FSM receives the EWMA output rather than the raw sensor reading, ensuring that alert decisions are based on the smoothed, validated signal rather than instantaneous noise. This architectural change eliminates the false debounce triggers that could occur in Lab 3.1 when a transient spike crossed the threshold.

- **FreeRTOS task architecture**: The three-task pipeline (acquisition, conditioning, display) operates with the same priority and synchronization structure as Lab 3.1, demonstrating that the conditioning pipeline integrates seamlessly into the existing real-time architecture without altering task timing or inter-task communication patterns.

4.2. Performance Analysis and System Limitations

The system operates within comfortable resource margins on the ATmega2560 platform. RAM usage of 29.7% (2 431 bytes of 8 192 bytes available) represents a moderate increase over Lab 3.1's 24.0% (1 969 bytes), with the additional consumption attributable to the two SignalConditioner instances maintaining circular buffers and EWMA state. The increase of approximately 462 bytes is consistent with the per-instance memory footprint: each SignalConditioner stores a 9-element float array (36 bytes), several float state variables, and index counters.

The median filter introduces a fill latency of 5 samples × 50 ms = 250 ms before the pipeline is fully primed. During this startup period, the median is computed over a partial window and the "Conditioned" flag in the STDIO report shows "FILLING". Once the window is full, the pipeline transitions to steady-state operation with no additional latency per sample — the median

computation (insertion sort on 5 elements) completes in microseconds, well within the 50 ms acquisition period.

The EWMA convergence time depends on the alpha parameter. With alpha = 0.3, the effective time constant is approximately $\tau \approx T_s/\alpha \approx 167$ ms, meaning the filter reaches 63% of a step change within roughly 3–4 acquisition cycles. Full convergence (within 1% of the final value) requires approximately $5\tau \approx 835$ ms. This responsiveness is appropriate for temperature monitoring where thermal time constants are on the order of seconds.

The LCD update rate of 500 ms remains adequate for human perception of temperature trends, and the 2-second STDIO report interval provides sufficient diagnostic resolution for verifying conditioning behavior during testing.

A notable limitation of the current implementation is that the conditioning parameters (median window size, EWMA alpha, saturation bounds) are compile-time constants defined in `sensor_data.h`. Changing these values requires recompilation and reflashing, which prevents runtime experimentation with different filter configurations. Additionally, the same conditioning parameters are applied to both sensor channels, whereas in practice the analog and digital sensors may benefit from different alpha values due to their distinct noise characteristics.

4.3. Proposed Improvements

Several enhancements could extend the system's capabilities and flexibility:

- **Runtime-adjustable parameters**: Implementing a serial command interface to modify the EWMA alpha and median window size at runtime would enable in-situ tuning without recompilation. Commands such as `SET ALPHA 0.5` or `SET WINDOW 7` could be parsed by a command handler task, allowing the operator to observe the effect of parameter changes on the conditioned output in real time.

- **Additional filter types**: The modular pipeline architecture could be extended with more sophisticated filters such as a Kalman filter (optimal for Gaussian noise when the process model is known), a Butterworth low-pass filter (for well-defined frequency-domain noise rejection), or an adaptive median filter that adjusts its window size based on detected noise levels.

- **Sensor fusion**: Combining the analog and digital sensor readings through a weighted averaging or voting scheme could improve overall measurement accuracy. When both sensors report similar temperatures, the fused estimate benefits from reduced variance; when they diverge significantly, the system could flag a sensor fault condition.

- **Data logging**: Adding an SD card module or circular buffer in EEPROM would enable offline analysis of temperature trends, conditioning pipeline behavior, and alert event history. Time-stamped logs would support post-hoc analysis of system performance and facilitate correlation of temperature events with external conditions.

- **Adaptive thresholds**: Rather than using fixed high and low thresholds, the system could implement a learning phase during startup that establishes a baseline temperature and automatically sets thresholds relative to this baseline (e.g., baseline + 5°C for the high threshold). This would make the system portable across different operating environments without manual threshold configuration.

4.4. Reflections on the Learning Experience

This laboratory work provided valuable insights into the importance of signal conditioning in embedded sensor systems. Working with raw sensor data in Lab 3.1 revealed the challenges of noise, quantization effects, and transient anomalies that can degrade measurement quality and trigger false alerts. The systematic application of a multi-stage conditioning pipeline in Lab 3.2 demonstrated how each processing stage addresses a specific class of signal impairment, and how the stages compose to produce a robust measurement chain.

The experience of implementing the median filter highlighted an important design trade-off: while the filter effectively removes impulsive noise, it introduces latency proportional to the window size and cannot track rapid legitimate changes faster than the window can shift. Similarly, the EWMA implementation illustrated the fundamental tension between responsiveness and stability — a trade-off that has no universal optimal solution but must be tuned for the specific application's requirements.

Designing the SignalConditioner as a reusable library with diagnostic getters for every pipeline stage proved essential for debugging and verification. Without the ability to inspect intermediate values (saturated, median, EWMA), it would have been difficult to determine whether unexpected behavior originated in the saturation bounds, the median window, or the EWMA tracking. This experience reinforces the principle that observability is a first-class design concern in embedded systems, not an afterthought.

4.5. Impact of Technology in Real-World Applications

Signal conditioning pipelines similar to the one implemented in this lab are fundamental to virtually all real-world sensor-based systems. Industrial process control systems apply multi-stage filtering to pressure, flow, and temperature transducers before feeding measurements to PID controllers, where noisy inputs would cause actuator chatter and mechanical wear. The pharmaceutical industry mandates validated signal conditioning for temperature sensors in clean rooms and autoclaves, where false readings could compromise product sterility or trigger unnecessary batch rejections.

Medical device monitoring represents a particularly critical application domain. Patient vital signs monitors apply sophisticated filtering to ECG, pulse oximetry, and temperature signals, where the consequences of both false alarms (alarm fatigue leading clinicians to ignore warnings) and missed alarms (undetected deterioration) can be life-threatening. The median-then-EWMA architecture implemented in this lab mirrors the general approach used in such systems, though

production medical devices employ additional stages such as baseline wander removal and artifact detection.

Automotive sensor systems provide another compelling example. Engine management units process signals from dozens of analog and digital sensors (throttle position, coolant temperature, manifold pressure) through conditioning pipelines that must balance rapid response for engine control with noise rejection for stable operation. The saturation stage is particularly relevant in automotive applications, where sensor faults (open circuit, short to ground) produce rail-voltage ADC readings that must be detected and clamped before downstream processing.

Environmental monitoring stations deployed for climate research or air quality assessment use signal conditioning to produce reliable long-term datasets from sensors exposed to harsh conditions. EWMA filtering with carefully chosen time constants enables these systems to capture meaningful environmental trends while rejecting measurement artifacts caused by precipitation, solar heating of sensor housings, or electromagnetic interference from nearby equipment. The techniques practiced in this laboratory work provide a solid foundation for understanding and implementing such real-world signal processing systems.

5. Note Regarding Usage of AI

During the preparation of this laboratory report, the author utilized artificial intelligence tools to enhance the quality and presentation of the documentation. Specifically, the following AI tools were employed:

- **Report Formatting:** Assistance with LaTeX document structure, formatting consistency, and adherence to academic writing standards.
- **Report Template Generation:** Creation of the initial document template with proper sectioning and organizational structure.
- **Content Rephrasing:** Refinement of technical descriptions and explanations to ensure grammatical accuracy and clarity without altering the technical substance.
- **Code Documentation:** Generation of inline comments within the source code to improve code readability and maintainability.
- **Development Environment Setup:** Configuration assistance for PlatformIO integration with the Wokwi simulation platform.

All AI-generated content was thoroughly reviewed, validated, and adjusted by the author to ensure technical accuracy, alignment with project requirements, and adherence to embedded systems best practices. The core technical implementation, circuit design, system architecture decisions, and experimental results represent the author's original work and understanding of the subject matter. The AI tools served as assistive technologies to enhance documentation quality and development workflow efficiency, while the author retained full responsibility for the content, correctness, and conclusions presented in this report.

6. Bibliography

[1] Bragarenco, A., Astafi, V., *Sisteme Electronice Încorporate: Indicaţii metodice pentru lucrări de laborator*, Universitatea Tehnică a Moldovei, Chişinău, 2024.

[2] Technical University of Moldova, *Embedded Systems: Practical Guide for Hardware-Software Interface Development*, Department of Microelectronics and Biomedical Engineering, Chişinău, 2024.

[3] Arduino, *Arduino Mega 2560 Rev3 Documentation*, `https://docs.arduino.cc/hardware/mega-2560/`, Accessed: 2026-03-14.

[4] Maxim Integrated, *DS18B20 — Programmable Resolution 1-Wire Digital Thermometer*, Datasheet, 2019.

[5] Smith, S.W., *The Scientist and Engineer's Guide to Digital Signal Processing*, California Technical Publishing, San Diego, 1997.

[6] PlatformIO, *Professional collaborative platform for embedded development*, `https://platformio.org/`, Accessed: 2026-03-14.

[7] Wokwi, *Online Electronics Simulator — Arduino, ESP32, Raspberry Pi Pico*, `https://wokwi.com/`, Accessed: 2026-03-14.

7. Appendix — Source Code

The complete source code for Laboratory Work 3.2 is organized below by architectural layer. All source files are available in the project GitHub repository.

GitHub Repository: `https://github.com/mcittkmims/es-labs`

7.1. Application Layer: Lab 3.2 Entry Point

lab3_2_main.h

Listing 7.1 *– lab3_2_main.h — Lab 3.2 Entry Point Interface*

```
% [Full source code - see GitHub repository]
% File: labs/lab/lab3_2/lab3_2_main.h
% Declares setup and loop functions for the dual-sensor temperature
% monitoring system with signal conditioning pipeline.
```

lab3_2_main.cpp

***Listing 7.2** – lab3_2_main.cpp — Lab 3.2 Entry Point Implementation*

```
% [Full source code - see GitHub repository]
% File: labs/lab/lab3_2/lab3_2_main.cpp
% Initializes STDIO, creates synchronization primitives, spawns
% three FreeRTOS tasks (acquisition, conditioning, display),
% and starts the scheduler.
```

7.2. Task Layer: Sensor Acquisition

task_acquisition.h

***Listing 7.3** – task_acquisition.h — Sensor Acquisition Task Interface*

```
% [Full source code - see GitHub repository]
% File: labs/lab/lab3_2/task_acquisition.h
% Declares the vTaskAcquisition() function for periodic sensor reading.
```

task_acquisition.cpp

***Listing 7.4** – task_acquisition.cpp — Sensor Acquisition Task Implementation*

```
% [Full source code - see GitHub repository]
% File: labs/lab/lab3_2/task_acquisition.cpp
% Periodically reads NTC thermistor (ADC) and DS18B20 (OneWire) at
% 50 ms intervals using vTaskDelayUntil(). Writes raw readings to
% shared g_sensorData under mutex and signals Task 2 via semaphore.
```

7.3. Task Layer: Signal Conditioning

task_conditioning.h

***Listing 7.5** – task_conditioning.h — Signal Conditioning Task Interface*

```
% [Full source code - see GitHub repository]
% File: labs/lab/lab3_2/task_conditioning.h
% Declares the vTaskConditioning() function for event-driven
% signal conditioning and threshold alerting.
```

task_conditioning.cpp

***Listing 7.6** – task_conditioning.cpp — Signal Conditioning Task Implementation*

```
% [Full source code - see GitHub repository]
% File: labs/lab/lab3_2/task_conditioning.cpp
% Applies three-stage conditioning pipeline (saturate -> median -> EWMA)
% to both sensor channels, feeds conditioned values to ThresholdAlert
```

```
% FSM, and controls LED indicators based on alert states.
```

7.4. Task Layer: Display and Reporting

task_display.h

Listing 7.7 *– task_display.h — Display and Reporting Task Interface*

```
% [Full source code - see GitHub repository]
% File: labs/lab/lab3_2/task_display.h
% Declares the vTaskDisplay() function for periodic LCD and
% STDIO report updates.
```

task_display.cpp

Listing 7.8 *– task_display.cpp — Display and Reporting Task Implementation*

```
% [Full source code - see GitHub repository]
% File: labs/lab/lab3_2/task_display.cpp
% Updates LCD with conditioned EWMA values and alert status (Page 0)
% and conditioning configuration (Page 1). Prints structured STDIO
% reports every 2 seconds showing raw, median, and EWMA values for
% both sensors along with alert states and system statistics.
```

7.5. Service Layer: Shared Data and Synchronization

sensor_data.h

Listing 7.9 *– sensor_data.h — Shared Data Structures and Pin Mapping*

```
% [Full source code - see GitHub repository]
% File: labs/lab/lab3_2/sensor_data.h
% Declares SensorReadings_t (with raw + conditioned fields),
% AlertStatus_t, FreeRTOS synchronization handles, hardware pin
% mapping, NTC/DS18B20 parameters, signal conditioning parameters
% (median window, EWMA alpha, saturation bounds), threshold alert
% parameters, and FreeRTOS task configuration constants.
```

sensor_data.cpp

Listing 7.10 *– sensor_data.cpp — Shared State Definitions*

```
% [Full source code - see GitHub repository]
% File: labs/lab/lab3_2/sensor_data.cpp
% Defines g_sensorData, g_alertData, xNewReadingSemaphore,
% xSensorMutex, and the sensorDataInit() function that creates
```

```
% the binary semaphore and mutex.
```

7.6. Service Layer: SignalConditioner

SignalConditioner.h

Listing 7.11 *– SignalConditioner.h — Signal Conditioning Pipeline Interface*

```
% [Full source code - see GitHub repository]
% File: labs/lib/SignalConditioner/SignalConditioner.h
% Declares the SignalConditioner class implementing a three-stage
% pipeline: saturation (clamping), median filter (sliding window
% with insertion sort), and EWMA (first-order IIR filter).
% All intermediate values accessible via getters for diagnostics.
% Maximum window size: 9 samples (fixed array, no heap allocation).
```

SignalConditioner.cpp

Listing 7.12 *– SignalConditioner.cpp — Signal Conditioning Pipeline Implementation*

```
% [Full source code - see GitHub repository]
% File: labs/lib/SignalConditioner/SignalConditioner.cpp
% Implements process(), reset(), saturate(), computeMedian(),
% and all getter methods. The median computation uses insertion
% sort on a local copy of the circular buffer contents.
% EWMA initializes to the first sample on first call.
```

7.7. Service Layer: ThresholdAlert

ThresholdAlert.h

Listing 7.13 *– ThresholdAlert.h — Hysteresis-Based Threshold Alert Interface*

```
% [Full source code - see GitHub repository]
% File: labs/lib/ThresholdAlert/ThresholdAlert.h
% Declares the ThresholdAlert class implementing a 4-state FSM:
% NORMAL -> DEBOUNCE_HIGH -> ALERT_ACTIVE -> DEBOUNCE_LOW -> NORMAL
% with configurable high/low thresholds and debounce count.
```

ThresholdAlert.cpp

Listing 7.14 *– ThresholdAlert.cpp — Hysteresis-Based Threshold Alert Implementation*

```
% [Full source code - see GitHub repository]
% File: labs/lib/ThresholdAlert/ThresholdAlert.cpp
% Implements the 4-state FSM for threshold detection with
```

```
% hysteresis and debounce filtering. Each transition requires
% N consecutive confirmations before state change.
```

7.8. Driver Layer: AnalogTempSensor

AnalogTempSensor.h

Listing 7.15 *– AnalogTempSensor.h — NTC Thermistor Driver Interface*

```
% [Full source code - see GitHub repository]
% File: labs/lib/AnalogTempSensor/AnalogTempSensor.h
% Declares the AnalogTempSensor class for reading temperature from
% an NTC thermistor via ADC with Steinhart-Hart Beta equation.
% Circuit: VCC --- [R_series] --- ADC_PIN --- [NTC] --- GND
```

AnalogTempSensor.cpp

Listing 7.16 *– AnalogTempSensor.cpp — NTC Thermistor Driver Implementation*

```
% [Full source code - see GitHub repository]
% File: labs/lib/AnalogTempSensor/AnalogTempSensor.cpp
% Implements ADC reading, resistance calculation from voltage
% divider equation, and temperature conversion using the
% Steinhart-Hart Beta parameter equation.
```

7.9. Driver Layer: DigitalTempSensor

DigitalTempSensor.h

Listing 7.17 *– DigitalTempSensor.h — DS18B20 OneWire Driver Interface*

```
% [Full source code - see GitHub repository]
% File: labs/lib/DigitalTempSensor/DigitalTempSensor.h
% Declares the DigitalTempSensor class for reading temperature
% from a DS18B20 sensor via the OneWire protocol.
```

DigitalTempSensor.cpp

Listing 7.18 *– DigitalTempSensor.cpp — DS18B20 OneWire Driver Implementation*

```
% [Full source code - see GitHub repository]
% File: labs/lib/DigitalTempSensor/DigitalTempSensor.cpp
% Implements OneWire-based temperature reading with non-blocking
% conversion support, device detection, and error handling for
% disconnected and power-on-reset conditions.
```

7.10. Driver Layer: LcdDisplay

LcdDisplay.h

***Listing 7.19** – LcdDisplay.h — LCD Display Driver Interface*

```
% [Full source code - see GitHub repository]
% File: labs/lib/LcdDisplay/LcdDisplay.h
% Declares the LcdDisplay class wrapping the LiquidCrystal_I2C
% library with convenience methods for two-line updates.
```

LcdDisplay.cpp

***Listing 7.20** – LcdDisplay.cpp — LCD Display Driver Implementation*

```
% [Full source code - see GitHub repository]
% File: labs/lib/LcdDisplay/LcdDisplay.cpp
% Implements LCD initialization, backlight control, and
% showTwoLines() method for atomic two-line display updates.
```

7.11. Driver Layer: Led

Led.h

***Listing 7.21** – Led.h — LED Driver Interface*

```
% [Full source code - see GitHub repository]
% File: labs/lib/Led/Led.h
% Declares the Led class for controlling a single LED with
% init(), turnOn(), turnOff(), and toggle() methods.
```

Led.cpp

***Listing 7.22** – Led.cpp — LED Driver Implementation*

```
% [Full source code - see GitHub repository]
% File: labs/lib/Led/Led.cpp
% Implements LED control using digitalWrite() with internal
% state tracking for toggle functionality.
```

7.12. Driver Layer: StdioSerial

StdioSerial.h

***Listing 7.23** – StdioSerial.h — STDIO Serial Redirect Interface*

```
% [Full source code - see GitHub repository]
% File: labs/lib/StdioSerial/StdioSerial.h
% Declares stdioSerialInit() for redirecting C stdio (printf)
% to the Arduino Serial interface.
```

StdioSerial.cpp

Listing 7.24 *– StdioSerial.cpp — STDIO Serial Redirect Implementation*

```
% [Full source code - see GitHub repository]
% File: labs/lib/StdioSerial/StdioSerial.cpp
% Implements STDIO redirection by assigning a custom putchar
% function to the AVR FILE stream, enabling printf() output
% over the hardware UART.
```